Red Hat build of Keycloak Administration and Security

Version 1.0.0, 2026-06-30

Home

Introduction

Course Title: Red Hat build of Keycloak Administration and Security

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Introduce Red Hat build of Keycloak
- Install and Configure Red Hat build of Keycloak
- Authentication and Authorization
- Identity Brokering and User Federation
- Red Hat build of Keycloak on Red Hat OpenShift
- Extending and Templating Keycloak
- Comprehensive Review

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience…
- Course or experience…
- Course or experience…

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Introduce Red Hat build of Keycloak

Introduce Red Hat build of Keycloak

Red Hat build of Keycloak is an open-source Identity and Access Management (IAM) solution designed to secure modern applications and services. It simplifies authentication and authorization by centralizing identity management, allowing developers to focus on business logic rather than security infrastructure.

Key features and concepts include:

- **Standardized Protocols:** Supports industry-standard identity protocols, including OpenID Connect (OIDC), OAuth 2.0, and SAML 2.0.
- **Security Mechanisms:** Implements robust security token mechanisms and credential isolation to protect sensitive user data.
- **Operational Architecture:** Provides a scalable architecture that manages user sessions and authentication flows effectively across distributed environments.
- **Unified Identity:** Acts as a centralized authority for user authentication and role-based or permission-based authorization.

Overview of Red Hat build of Keycloak features

Overview of Red Hat build of Keycloak features

Red Hat build of Keycloak is an open-source identity and access management (IAM) solution designed for modern applications and services. It provides a centralized security layer that allows developers and administrators to secure their applications with minimal effort, eliminating the need to write custom authentication or authorization code.

Core Features

Red Hat build of Keycloak offers a comprehensive suite of features designed to handle complex identity requirements at scale:

- **Single Sign-On (SSO):** Users authenticate once and gain access to multiple applications. This reduces password fatigue and simplifies the user experience across an organization's ecosystem.
- **Identity Brokering:** Keycloak acts as an intermediary, allowing users to authenticate via external third-party identity providers (such as Google, GitHub, or enterprise LDAP/Active Directory) while maintaining a consistent internal identity.
- **User Federation:** Keycloak can bridge existing user stores. You can import users from external LDAP or Active Directory systems or keep them synchronized, allowing for seamless integration with legacy enterprise infrastructure.
- **Standard-Based Security:** It is built on proven industry standards to ensure interoperability and robust security practices. Supported protocols include:

- **OpenID Connect (OIDC):** An identity layer on top of OAuth 2.0.
- **OAuth 2.0:** A framework for authorization.
- **SAML 2.0:** An XML-based standard for exchanging authentication and authorization data.
- **Centralized Management:** Through the Admin Console and Admin CLI, administrators have a single pane of glass to manage users, roles, sessions, and security policies across all connected applications.
- **Customizable User Experience:** Keycloak provides a templating engine for the login, registration, and account management pages, allowing organizations to align the UI with their corporate branding.
- **Fine-Grained Authorization:** Beyond simple authentication, Keycloak supports Role-Based Access Control (RBAC) and attribute-based access control, enabling developers to enforce sophisticated security policies at the application level.

Why use Red Hat build of Keycloak?

By delegating the security burden to Red Hat build of Keycloak, your applications benefit from:

1. **Credential Isolation:** Applications never handle user credentials directly; they rely on tokens issued by Keycloak.
2. **Rapid Time-to-Market:** Focus on building business logic rather than complex authentication flows.
3. **Enhanced Security:** Benefit from centralized auditing, brute-force protection, and consistent security updates provided by Red Hat.
4. **Operational Flexibility:** Whether running in bare-metal environments or containerized platforms like Red Hat OpenShift, Keycloak is designed for portability and high availability.

Red Hat build of Keycloak is designed to be highly extensible. If your organization has unique requirements—such as custom token mappers, proprietary authentication protocols, or unique user storage backends—the Keycloak Service Provider Interface (SPI) allows you to extend the platform functionality to meet these specific needs.

Knowledge Check: Keycloak Systems

1. **True or False:** Red Hat build of Keycloak stores user credentials directly within the application database to ensure higher performance.
2. **Multiple Choice:** Which of the following is an identity layer built on top of OAuth 2.0 supported by Keycloak?
    - A) SAML 1.0
    - B) OpenID Connect (OIDC)
    - C) RADIUS
    - D) LDAP

Answers: 1. **False. Keycloak promotes credential isolation; applications work with tokens**

rather than storing or handling user credentials. 2. **B.**

Single Sign-On standards (OIDC, OAuth 2.0, SAML 2.0)

Single Sign-On standards (OIDC, OAuth 2.0, SAML 2.0)

In the Red Hat build of Keycloak, enabling Single Sign-On (SSO) relies on standardized protocols that allow applications to delegate authentication to a centralized server. Understanding the distinctions between these protocols is critical for architecting secure applications.

OpenID Connect (OIDC) and OAuth 2.0

While often discussed together, OAuth 2.0 and OpenID Connect serve distinct purposes:

- **OAuth 2.0:** Primarily an **authorization framework**. It allows an application to obtain an "access token" to invoke services on behalf of a user. It does not natively provide identity information about the user.
- **OpenID Connect (OIDC):** An **identity layer** built on top of the OAuth 2.0 protocol. It adds an "ID token" to the OAuth 2.0 flow, which contains information about the authenticated user.

Key Characteristics of OIDC

- **Web-Optimized:** Designed specifically for web-based ecosystems.
- **Lightweight:** Uses JSON for tokens, making them highly compatible with JavaScript-based client-side applications (e.g., HTML5/SPA).
- **Developer Experience:** Includes advanced security features, such as the `iframe` mechanism used to determine a user's current login status without forcing a full redirect.

SAML 2.0

Security Assertion Markup Language (SAML) 2.0 is a mature, XML-based standard for exchanging authentication and authorization data between parties.

Key Characteristics of SAML

- **Architecture:** It acts as a layer on top of the web, originally derived from SOAP and web service messaging.
- **Verbosity:** SAML is significantly more verbose than OIDC, as it relies on complex XML documents for communication.
- **Security Mechanisms:** It utilizes XML signatures and encryption to verify the integrity and confidentiality of requests and responses.
- **Use Cases:**
- **Authentication Delegation:** An application requests that the Keycloak server authenticates a user. Upon success, Keycloak returns an XML document containing an "SAML Assertion" with user attributes and role mappings.
- **Legacy Integration:** Often chosen for existing enterprise applications already secured with

SAML, or in environments where its maturity is a strict compliance requirement.

Comparison Summary

Feature	OpenID Connect (OIDC)	SAML 2.0
Format	JSON	XML
Primary Use	Modern Web/Mobile Applications	Enterprise/Legacy Applications
Complexity	Lower (Developer-friendly)	Higher (Verbose/Complex)
Maturity	High (Modern standard)	Very High (Legacy standard)

Practical Considerations

When choosing between these standards for your Red Hat build of Keycloak deployment, consider the following:

1. **Client-Side Compatibility:** If you are building modern JavaScript or single-page applications, OIDC is the preferred choice due to its native JSON support and ease of implementation.
2. **Existing Infrastructure:** If your environment already utilizes SAML for existing applications, Keycloak acts as an effective SAML Identity Provider (IdP) to maintain security posture without refactoring legacy authentication logic.
3. **Credential Isolation:** In both protocols, Red Hat build of Keycloak ensures credential isolation by acting as the broker; the end application never sees the user's password, only the token or assertion confirming the identity.

For most new projects in Red Hat build of Keycloak, default to **OIDC** unless there is a specific requirement for SAML-based interoperability.

Architecture and operational concepts

Architecture and Operational Concepts

Red Hat build of Keycloak is designed as a high-performance, identity and access management solution that decouples application security logic from the application code itself. By leveraging standard protocols, it acts as an Identity Provider (IdP) that centralizes the management of user identities, roles, and permissions.

Core Architectural Components

The architecture of Red Hat build of Keycloak is built upon a modular, scalable foundation designed to handle complex authentication and authorization requirements.

- **Realms:** The fundamental boundary for configuration. A realm is a space where you manage a set of users, credentials, roles, and groups. A user belongs to and logs into a realm. Realms are isolated from one another; for example, an organization can host multiple realms to separate distinct business units or B2B partners.
- **Clients:** These are the entities that request the authentication of a user. Examples include web applications, mobile apps, or backend services. Clients rely on the Keycloak server to verify identity and issue security tokens.

- **Users:** Entities that have the ability to log into your system. Users can be managed directly within the local Keycloak database, or their identity can be federated from external stores like LDAP or Active Directory.
- **Identity Brokering:** Allows Keycloak to act as an intermediary, delegating authentication to external Identity Providers (e.g., GitHub, Google, or other OIDC/SAML providers) while maintaining a unified identity context for your applications.

Operational Concepts

Understanding how Red Hat build of Keycloak operates requires a look at how it manages the flow of trust and the lifecycle of data.

Multi-Tenancy and Organizations

As defined in the context of organizational management, Red Hat build of Keycloak enables multi-tenancy within a realm through the **Organization** construct. * **Isolation:** While realms provide the primary scope, organizations allow for granular management of members within a realm. * **B2B Integration:** Organizations provide a dedicated context for business partners or customers, allowing administrators to manage the lifecycle of these external identities without polluting the primary user directory.

Security Token Mechanisms

Keycloak operates primarily through the issuance of security tokens (JWTs - JSON Web Tokens). * **Credential Isolation:** Keycloak ensures that credentials (passwords, OTPs, or biometric data) never leave the server. Applications interact only with the token, which contains claims about the user (e.g., identity, roles, organization membership). * **Token Propagation:** Organizations can propagate organization-specific claims directly into these tokens, enabling downstream applications to make authorization decisions based on the user's specific context within that organization.

Policy-Based Configuration

The system utilizes a modular **Client Policy** framework to enforce security postures. This is structured around four pillars: 1. **Condition:** The specific trigger or criteria that must be met (e.g., the client is using a specific protocol). 2. **Executor:** The action taken when a condition is met (e.g., forcing a specific authentication flow). 3. **Profile:** A collection of policies that define a security standard for a set of clients. 4. **Policy:** The pairing of conditions with executors to enforce uniform security rules across the ecosystem.

Operational Workflow Summary

Component	Operational Function
Authentication	Validates the user's identity through credentials or brokered external providers.
Authorization	Determines what the authenticated user is allowed to do via roles and permissions.
Token Service	Issues OIDC/OAuth 2.0/SAML tokens that secure communication between the user and the client application.

Component	Operational Function
Management	Administers the lifecycle of users, groups, and organizations via the Admin Console or CLI.

By centralizing these functions, Red Hat build of Keycloak reduces the operational overhead of securing multiple applications, providing a single source of truth for identity data while maintaining strict security boundaries between distinct user segments.

Security token mechanisms and credential isolation

Security Token Mechanisms and Credential Isolation

In distributed architectures, managing identity securely across disparate trust domains requires robust token mechanisms. Red Hat build of Keycloak leverages industry-standard protocols to ensure that credentials remain isolated and that access is strictly controlled, even when services operate under different security authorities.

Token Mechanisms and Credential Integrity

At the core of Keycloak's security model is the issuance and validation of tokens. To prevent the misuse of credentials, Keycloak adheres to the principle of **Credential Isolation**, ensuring that a service receiving a token cannot use that same token to impersonate the user or client against a different, unauthorized service.

Proof of Possession (PoP)

A critical mechanism for preventing token theft and replay attacks is **Proof of Possession**.

- **Mechanism:** When a client requests a token, it cryptographically signs the request using its private key.
- **Validation:** The Resource Server receives the token, which contains a fingerprint of the client's public key. The server validates the request by checking the client's signature against this embedded fingerprint.
- **Result:** Even if an attacker intercepts the token, they cannot use it because they lack the private key required to sign the requests, effectively binding the token to the legitimate owner.

OAuth Identity and Authorization Chaining

When applications must access resources distributed across multiple trust domains—each with its own authorization server—Keycloak supports **OAuth Identity and Authorization Chaining**. This approach prevents the need to share long-lived user credentials across domains.

The Chaining Flow

The process combines RFC 8693 (Token Exchange) and RFC 7523 (JWT Profile for Client Authentication) to bridge security domains:

1. **Discovery:** The client in Trust Domain A identifies the Authorization Server (AS) of Trust

Domain B.

2. **Exchange:** The client presents its current token to the AS in Domain A.
3. **Grant Issuance:** Domain A's AS returns a JWT authorization grant designed specifically for Domain B.
4. **Presentation:** The client presents this JWT grant to the AS in Domain B.
5. **Token Issuance:** Domain B validates the grant and issues a localized access token.
6. **Resource Access:** The client uses the new access token to interact with the protected resource in Domain B.

This mechanism ensures that the access token granted by Domain B has **limited privileges**. It is scoped specifically to the resources within that domain, ensuring that if Domain B were compromised, the stolen token could not be leveraged to access resources in Domain A or other external services.

Hands-on Activity: Observing Token Scoping

In this exercise, you will examine how tokens are restricted to specific audiences to ensure isolation.

1. **Step 1: Access the Admin Console** Log into your Keycloak instance and navigate to the **Clients** section.
2. **Step 2: Configure Client Scopes** Select a client and navigate to the **Client Scopes** tab. Note how specific claims are attached to the token.
3. **Step 3: Test Token Request** Using the `kcadm.sh` CLI or a REST client, request a token for the client. [source,bash] ---- # Example: Requesting a token via CLI ./kcadm.sh get tokens --realm myrealm --client myclient --user myuser ----
4. **Step 4: Decode and Verify** Use a tool like `jwt.io` to decode the generated access token. Observe the `aud` (audience) claim.
   - **Question:** Is the audience restricted to the intended resource server, or is it overly permissive?
5. **Step 5: Review Credential Isolation** Observe that the token contains no raw user credentials (like passwords). This isolation ensures that even if the token is logged or intercepted at the Resource Server, the user's master credentials remain entirely shielded within the Keycloak identity provider.

Quiz: Describe Keycloak Systems

Quiz: Describe Keycloak Systems

Test your understanding of the core architectural concepts and the foundational role of the Red Hat build of Keycloak in modern identity and access management (IAM) environments.

Which of the following best describes the primary function of Red Hat build of Keycloak in a

distributed application architecture?

- A) It serves as a primary database for storing application business logic.
- B) It acts as a centralized Identity and Access Management (IAM) server, providing Single Sign-On (SSO) and identity brokering.
- C) It is a network firewall designed to block unauthorized traffic at the edge of the data center.
- D) It is a web server framework used to build microservices APIs.
- B

Red Hat build of Keycloak is an open-source IAM solution that enables SSO, identity brokering, and user federation, allowing applications to delegate authentication to a central authority rather than managing credentials independently.

Red Hat build of Keycloak supports several industry-standard protocols for secure authentication and authorization. Which of the following is NOT a protocol natively supported by Keycloak?

- A) OIDC (OpenID Connect)
- B) OAuth 2.0
- C) SAML 2.0
- D) SSH (Secure Shell)
- D

Red Hat build of Keycloak is built on open standards such as OIDC, OAuth 2.0, and SAML 2.0. SSH is a cryptographic network protocol used for secure remote login to computers, which falls outside the scope of web-based identity management.

In the context of Keycloak architecture, what is meant by "Credential Isolation"?

- A) Storing user passwords in plain text for easier retrieval by developers.
- B) Ensuring that individual applications do not handle or store user credentials, as authentication is offloaded to the Keycloak server.
- C) Physically moving the server to an isolated network segment without internet access.
- D) Forcing all users to reset their passwords every 24 hours.
- B

Credential isolation is a critical security benefit of centralized IAM. By using protocols like OIDC, applications interact with the Keycloak server to verify identities, ensuring that sensitive user credentials remain sequestered within the secure Keycloak environment and are never exposed to the client applications themselves.

Which architectural concept allows Keycloak to bridge multiple identity sources (like LDAP, Active Directory, or Social Logins) and present them as a unified identity?

- A) Identity Brokering
- B) Load Balancing
- C) Database Replication
- D) Reverse Proxying

- A

Identity Brokering enables the integration of external identity providers (IdPs). This allows Keycloak to act as an intermediary, delegating authentication to third-party systems or internal directories while providing a consistent SSO experience to the end user.

Quiz: Introduce Red Hat build of Keycloak

Quiz: Introduce Red Hat build of Keycloak

Test your knowledge of the fundamental concepts and architectural pillars of Red Hat build of Keycloak.

Which of the following protocols are natively supported by Red Hat build of Keycloak for identity and access management? (Select all that apply)

- ☑ OpenID Connect (OIDC)
- ☑ OAuth 2.0
- ☑ SAML 2.0
- ☐ Kerberos (native implementation)
- ☐ SSH

Red Hat build of Keycloak is designed to function as an identity provider supporting industry-standard protocols, specifically OIDC (built on top of OAuth 2.0) and SAML 2.0, allowing seamless integration with modern applications and legacy systems.

What is the primary purpose of the Red Hat build of Keycloak's architectural design regarding credential isolation?

- ☐ To allow applications to store user passwords locally.
- ☐ To ensure that the identity provider manages credentials, keeping them isolated from individual application databases.
- ☐ To force users to re-authenticate for every single microservice.
- ☐ To replace the need for an underlying database.

Credential isolation is a core security benefit. By centralizing authentication, applications never handle raw user credentials, which significantly reduces the attack surface and

simplifies compliance requirements.

True or False: Red Hat build of Keycloak 26.6 provides configuration options to optimize for faster startup times and reduced memory footprints.

- [x] True
- [ ] False

As noted in the configuration documentation, administrators can apply specific build options and configuration methods to tailor the runtime environment for performance, ensuring the service is optimized for the specific requirements of the host environment.

Which concept best describes Red Hat build of Keycloak's role in a Single Sign-On (SSO) environment?

- [ ] A database backup tool for web applications.
- [ ] A central identity provider that manages authentication and authorization across multiple disparate applications.
- [ ] A load balancer that distributes traffic across multiple web servers.
- [ ] A firewall that blocks all incoming requests from the public internet.

Red Hat build of Keycloak acts as a central identity provider (IdP). It decouples the authentication logic from the application, allowing users to authenticate once and access multiple authorized services without re-entering credentials.

Install and Configure Red Hat build of Keycloak

Install and Configure Red Hat build of Keycloak

Installing and configuring the Red Hat build of Keycloak involves setting up the server environment, accessing the administrative interfaces, and defining initial security parameters.

Key aspects include:

- **Development Mode:** Utilizing the `start-dev` command for rapid deployment and testing in containerized environments, such as Podman.
- **Security Requirements:** Implementing secure configuration practices to transition from development to production environments.
- **Admin Console:** Using the web-based interface to manage realms, clients, and security settings.
- **Admin CLI:** Leveraging command-line tools to automate configuration tasks and perform administrative operations.
- **Deployment:** Integrating the server into target environments, including containerized deployments on platforms like Red Hat OpenShift.

Development mode installation requirements

Development mode installation requirements

Red Hat build of Keycloak provides a "development mode" specifically designed for rapid prototyping, theme development, and initial exploration. This mode prioritizes ease of use and immediate feedback by relaxing several security and performance constraints required for production environments.

Development Mode Overview

Development mode is the recommended starting point when you want to get an instance up and running quickly. Unlike production mode, which enforces strict security and networking configurations, development mode automatically applies a set of convenient defaults that facilitate a smoother development lifecycle.

Development mode should **never** be used in production environments. It is intended strictly for local development and testing purposes.

Default Configuration Characteristics

When Red Hat build of Keycloak is started in development mode, the server initializes with the following configuration defaults:

- **HTTP Enabled:** The server enables non-TLS HTTP communication by default. This avoids the

requirement to configure SSL/TLS certificates and keystores during the initial setup phase.

- **Disabled Strict Hostname Resolution:** Strict hostname checks are disabled. This allows you to access the server using various hostnames (such as `localhost` or local IP addresses) without requiring complex DNS or proxy configuration.
- **Local Caching:** The caching mechanism is set to `local`. This means no distributed cache is used, which simplifies the startup process but prevents high availability or clustering capabilities.
- **Theme and Template Caching Disabled:** Theme and template caching are turned off. This is a crucial feature for developers, as it allows changes to themes or FreeMarker templates to take effect immediately without requiring a server restart.

System Prerequisites

Before launching in development mode, ensure your environment meets the following requirements:

1. **Supported Java Version:** You must have a compatible version of the Java Runtime Environment (JRE) or Java Development Kit (JDK) installed. Refer to the **Red Hat Knowledgebase Article on supported configurations** to verify the specific versions supported by your version of Red Hat build of Keycloak.
2. **Installation Directory:** Ensure you have downloaded and extracted the distribution archive. Familiarize yourself with the directory structure under the installation root to understand where configuration files and deployment artifacts reside.
3. **Port Availability:** By default, the server will attempt to bind to standard ports (typically 8080). Ensure these ports are not occupied by other services on your local machine.

Guided Exercise: Preparing for Development Mode

Follow these steps to verify your local environment:

1. **Check Java Version:** Open your terminal and run the following command to verify your installation:

   ```
   java -version
   ```

2. **Validate Installation Directory:** Navigate to the folder where you extracted the Red Hat build of Keycloak distribution. Ensure you have the `bin/`, `conf/`, and `providers/` directories visible.
3. **Consult Supported Configurations:** Before proceeding with the installation, visit the Red Hat Knowledgebase to confirm that your specific OS, Java version, and database (if you intend to override the default H2 database) are supported.

If you are developing custom themes, remember that development mode's automatic cache disabling is your best tool. If you decide to transition to production mode later, you will need to manually configure these cache settings to optimize performance.

Admin Console exploration and configuration

Admin Console exploration and configuration

The Red Hat build of Keycloak Admin Console provides a centralized web-based user interface for managing all aspects of your identity and access management system. From this interface, administrators can manage realms, users, clients, identity providers, and authentication flows.

Understanding the Admin Console

The Admin Console is the primary interaction point for configuring Keycloak security policies. Before accessing the console, ensure you have completed the initial installation and created your initial administrator account.

If no administrators exist, you must follow the `Creating the first administrator` procedure to establish the root account before you can log in to the Admin Console.

Accessing the Admin Console

To sign in, navigate to the base URL of your Keycloak instance and append `/admin` to the address.

- **Sign In:** Provides authentication for administrative access.
- **Permissions:** If you are not a master-realm administrator, ensure an existing administrator has granted you the necessary privileges to manage specific realms.

Configuring Hostname for the Admin Console

In production environments, you may often place Keycloak behind a reverse proxy. To ensure the Admin Console remains reachable when using a different address than the public-facing hostname, use the `hostname-admin` configuration option.

```
# Example of setting the admin console hostname via CLI
./kc.sh start --hostname-admin=admin.example.com
```

- **Type:** String
- **CLI Option:** `--hostname-admin`
- **Description:** Defines the specific address for accessing the administration console. Use this when the management interface is exposed on a distinct network path or proxy configuration.

Guided Exercise: Explore and Configure the Admin Console

In this exercise, you will log in to the Admin Console and familiarize yourself with the navigation structure.

Step 1: Access the Console

1. Open your web browser and navigate to the URL where your Keycloak instance is running (e.g., `http://localhost:8080/admin`).
2. Enter the credentials created during your initial setup.

Step 2: Navigate the Realm Menu

1. Observe the **Master** realm selector in the top-left corner.
2. Click the dropdown to see how you can switch between different isolated security environments.
3. Select "Create Realm" to understand the basic metadata required for a new environment (e.g., Realm Name, Display Name).

Step 3: Explore Configuration Modules

Navigate through the sidebar to identify the primary management areas: * **Manage:** Contains Users, Groups, Roles, and Clients. * **Configure:** Houses Realm Settings, Authentication, Identity Providers, and User Federation.

Step 4: Verify Admin Permissions

1. Navigate to **Users** in the sidebar.
2. Select your current administrative user.
3. Click on the **Role Mapping** tab to view the fine-grained permissions currently assigned to your account.

For more information on managing these permissions, refer to the **Admin Console Access Control** chapter in the official documentation.

Troubleshooting Access Issues

- **Authentication Failure:** Verify if your account is locked due to excessive failed attempts or if your permissions were revoked in the master realm.
- **Reverse Proxy Issues:** If the page fails to load after a successful login, verify that your `--hostname-admin` matches the external URL provided by your load balancer or reverse proxy.

Admin CLI operations

Admin CLI operations

The Red Hat build of Keycloak provides a powerful command-line interface (CLI) that allows administrators to automate tasks, manage resources, and perform administrative operations without interacting with the graphical Admin Console. This approach is essential for DevOps pipelines, bulk configuration management, and scripting repetitive tasks.

Understanding the Admin CLI

The Admin CLI is a wrapper that facilitates interaction with the Keycloak Admin REST API. When you execute commands via the CLI, the tool translates these instructions into authenticated HTTP requests sent to the Admin REST endpoints.

Key characteristics include: * **REST API Interaction:** Every CLI action corresponds to an endpoint operation. * **Credential Isolation:** The CLI manages authentication contexts, ensuring that only authorized users can modify server configurations. * **Scriptability:** Because it is command-based, it is ideal for integration into CI/CD workflows and automated provisioning scripts.

Prerequisites for CLI Operations

Before performing operations, you must ensure: 1. **Authentication:** You have authenticated as a user with appropriate administrative permissions. 2. **Connectivity:** The CLI tool has network access to the Keycloak server instance. 3. **Configuration:** The server URL and realm context are correctly specified.

Common CLI Operations

The Admin CLI supports a broad spectrum of administrative tasks. Common operations include:

- **User Management:** Creating, listing, updating, or deleting user accounts.
- **Client Management:** Registering new OIDC clients or configuring existing service accounts.
- **Realm Configuration:** Modifying realm settings and security policies.
- **Role and Group Mapping:** Managing access control assignments programmatically.

Access to Admin REST endpoints is strictly controlled. Ensure that the service account or user account used by the CLI has the necessary roles assigned via the Admin Console's **Access Control** settings.

Guided Exercise: Interacting with the Admin CLI

In this exercise, you will authenticate with the server and list existing users.

Prerequisites

- Ensure your Red Hat build of Keycloak server is running.
- Ensure the `kcadm.sh` (or `kcadm.bat`) script is available in your `bin/` directory.

Step-by-Step Instructions

1. **Authenticate to the Server:** Use the `config credentials` command to establish an authenticated session.

```
./kcadm.sh config credentials --server http://localhost:8080/ --realm master --user
admin --password <your-password>
```

2. **Verify Connectivity:** List the current realms to ensure your authentication was successful.

```
./kcadm.sh get realms
```

3. **List Users in a Specific Realm:** Retrieve a list of users for a realm (e.g., `my-realm`).

```
./kcadm.sh get users -r my-realm
```

4. **Create a New User:** Use the `create` command to add a new user via a JSON input file or inline attributes.

```
./kcadm.sh create users -r my-realm -s username=newuser -s enabled=true
```

Troubleshooting Common CLI Issues

- **"401 Unauthorized":** This usually indicates that the session token has expired or the user lacks the required `realm-admin` permissions. Re-authenticate using the `config credentials` command.
- **Connection Refused:** Ensure the `--server` URL matches the protocol (http/https), hostname, and port where your Keycloak instance is listening.
- **Missing JSON Attributes:** When using the `create` or `update` commands, ensure all mandatory fields required by the REST API are provided. Use the `get` command on an existing object to see the expected schema.

Guided Exercise: Red Hat build of Keycloak Installation

Guided Exercise: Red Hat build of Keycloak Installation

In this exercise, you will perform a development mode installation of the Red Hat build of Keycloak. This process establishes the foundational environment required to begin exploring the Admin Console and identity management capabilities.

Prerequisites

- A system with Java 17 or higher installed.
- The Red Hat build of Keycloak distribution archive (downloaded from the Red Hat Customer Portal).
- Sufficient permissions to create files and execute processes in your target directory.

Step-by-Step Installation

Follow these steps to install and start the Red Hat build of Keycloak in development mode.

1. Extract the distribution

Navigate to the directory where you downloaded the archive and extract the contents.

```
tar -xvf rhbk-26.6.0.tar.gz
cd rhbk-26.6.0
```

2. Understand the Development Mode

By default, running Keycloak in development mode enables features that simplify initial setup, such as: * HTTP is enabled by default (no TLS required for initial testing). * The Admin Console is accessible via `localhost`.

Development mode is intended strictly for local development and testing. Do not use this configuration in production environments, as it lacks secure defaults like HTTPS and production-grade database persistence.

3. Start the Server

To launch the server, execute the `kc.sh` script (or `kc.bat` on Windows) located in the `bin` directory with the `start-dev` flag.

```
bin/kc.sh start-dev
```

You will see logs indicating that the server is initializing the underlying WildFly-based framework and preparing the H2 database for in-memory storage.

4. Verify Server Readiness

Once the logs display `Keycloak 26.6.0 started in (DEVELOPMENT) mode`, the server is ready to accept connections. Open your web browser and navigate to the following URL:

`http://localhost:8080/`

5. Create the Initial Admin User

Since this is a fresh installation, you must create the primary administrator account before you can access the Admin Console.

1. Click on the **Administration Console** link on the welcome page.
2. Enter the following details for the initial user:
    - **Username:** `admin`
    - **Password:** `your_secure_password`
    - **Password Confirmation:** `your_secure_password`
3. Click **Create**.
4. Once created, click the **Back to Admin Console** link to log in using the credentials you just

established.

Troubleshooting

If the server fails to start, verify the following:

- **Port Conflicts:** Ensure that port `8080` is not already in use by another service on your machine.
- **Java Version:** Run `java -version` to ensure you are using a supported version (JDK 17+).
- **Logs:** Check the `data/log/keycloak.log` file for specific stack traces or bind errors.

Summary

You have successfully installed the Red Hat build of Keycloak in development mode and initialized the master realm administrator. You are now prepared to proceed to the next module: **Explore and Configure the Admin Console**.

Guided Exercise: Explore and Configure the Admin Console

Guided Exercise: Explore and Configure the Admin Console

In this exercise, you will navigate the Red Hat build of Keycloak Admin Console. The Admin Console is the centralized web-based interface used to manage realms, clients, users, and security policies.

Prerequisites

- Red Hat build of Keycloak is installed and running.
- You have created an initial administrator account (see "Creating the first administrator" if you have not yet initialized the system).

Objectives

- Access and authenticate to the Admin Console.
- Navigate the primary navigation menu.
- Inspect existing realm configurations.
- Understand the scope of administrative permissions.

Procedure

Step 1: Access the Admin Console

1. Open your web browser and navigate to the base URL of your Keycloak instance (e.g., `http://localhost:8080`).
2. Click the **Administration Console** link.
3. Authenticate using the credentials for the initial administrator account you created during the installation phase.

Step 2: Navigating the Dashboard

Once logged in, you are greeted by the **Dashboard**. This screen provides a high-level overview of the currently selected realm.

- **Realm Selector:** Located in the top-left corner, use this to switch between the `master` realm and any other realms you have created.
- **Navigation Menu:** Located on the left sidebar, this menu categorizes administrative tasks into:
- **Configure:** Manage realms, clients, identity providers, and authentication flows.
- **Manage:** Perform operations on users, groups, and sessions.
- **Events:** Monitor system-wide events and user activity.

Step 3: Inspecting Realm Settings

1. In the sidebar, click **Realm settings**.
2. Review the tabs available:
    - **General:** View basic realm information and display names.
    - **Login:** Configure registration, email verification, and "forgot password" workflows.
    - **Themes:** Manage the visual appearance of the login and account pages.
    - **Tokens:** Configure settings for OIDC and SAML token lifespans and signatures.

Step 4: Configuring Administrative Access

Managing who can access the Admin Console is critical for security.

1. Navigate to the **Users** section in the left sidebar.
2. Select your administrator user.
3. Click on the **Role mapping** tab.
4. Verify the roles assigned to your user. The `realm-admin` role is typically required to manage the configurations you explored in the previous steps.

If you need to delegate administration, navigate to the **Manage** > **Users** section to create a new user, and then assign specific administrative roles via the **Role mapping** tab. For detailed information on granular access control, refer to the **Admin Console Access Control** documentation.

Verification

To ensure your environment is configured correctly: 1. Attempt to create a test user under the `master` realm. 2. If successful, navigate to the **User list** to confirm the user appears. 3. Confirm that you can toggle between the `master` realm and a custom realm (if created) using the top-left dropdown menu.

Troubleshooting

- **Issue:** "Access Denied" or missing menu items.
- **Resolution:** Ensure your account has the necessary realm-level roles assigned. You may need to log in with the initial super-administrator account to audit your permissions.
- **Issue:** Cannot see the "Manage" menu.
- **Resolution:** Verify that you have a realm selected in the Realm Selector. If the selector shows "Select a realm," the Admin Console will not display the full suite of management options.

Guided Exercise: Red Hat build of Keycloak Admin CLI

Guided Exercise: Red Hat build of Keycloak Admin CLI

In this exercise, you will interact with the Red Hat build of Keycloak using the `kcadm.sh` (or `kcadm.bat`) Admin CLI tool. The Admin CLI is a powerful utility that allows you to perform administrative tasks, such as creating realms, managing users, and updating configurations, directly from the terminal.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- The `kcadm` tool installed (located in the `bin/` directory of your Keycloak distribution).
- Credentials for an administrative user.

Step 1: Authenticate with the Admin CLI

Before running commands, you must establish an authenticated session. The CLI uses an OIDC session to perform operations against the Admin REST API.

1. Open your terminal and navigate to your Keycloak `bin/` directory.
2. Run the authentication command, providing your server URL and credentials:

```
./kcadm.sh config credentials --server http://localhost:8080/ --realm master --user
admin --password <YOUR_PASSWORD>
```

Note: If authentication is successful, the CLI saves an access token locally, allowing subsequent commands to run without re-authenticating.

Step 2: Explore Realms

Once authenticated, you can list the realms currently managed by your Keycloak instance.

1. Execute the following command to retrieve a list of realms:

```
./kcadm.sh get realms
```

1. Identify the JSON output for the `master` realm. This confirms that your CLI session is correctly communicating with the server.

Step 3: Create a New Realm via CLI

The Admin CLI allows for the automation of resource creation by passing JSON configuration files.

1. Create a file named `new-realm.json` with the following content:

```
{
    "realm": "Development",
    "enabled": true
}
```

1. Run the following command to create the realm:

```
./kcadm.sh create realms -f new-realm.json
```

1. Verify the creation by listing the realms again:

```
./kcadm.sh get realms
```

Step 4: Manage Users via CLI

You can also perform CRUD operations on users. Let's create a new user in your newly created "Development" realm.

1. Create a user:

```
./kcadm.sh create users -r Development -s username=devuser -s enabled=true
```

1. Set a password for this user:

```
./kcadm.sh set-password -r Development --username devuser --new-password
MySecurePassword123!
```

Step 5: Troubleshooting and Verification

If you encounter issues, verify the following:

- **Connectivity:** Ensure the server is reachable via the `--server` flag.
- **Permissions:** The user account used for authentication must have the `manage-realm` or appropriate fine-grained permissions for the targeted realm.
- **Debug Mode:** If you are receiving unexpected errors, append `--debug` to your command to see

the underlying HTTP requests and responses:

```
./kcadm.sh get realms --debug
```

Summary

You have successfully: * Authenticated a CLI session against the Keycloak Admin API. * Retrieved and created resources using the `get` and `create` operations. * Managed user credentials via the command line, demonstrating how the Admin CLI can be used for automation in CI/CD pipelines or operational scripting.

Lab: Install Red Hat build of Keycloak

Lab: Install Red Hat build of Keycloak

In this lab, you will perform a containerized deployment of the Red Hat build of Keycloak in development mode. This environment is intended for learning, experimentation, and initial configuration exploration.

Prerequisites

- A system with `podman` or `docker` installed.
- Network access to the Red Hat Ecosystem Catalog (`registry.redhat.io`).
- Sufficient permissions to pull container images and bind to local network ports (8080).

Scenario

You are tasked with setting up a local instance of Red Hat build of Keycloak to begin exploring the administrative interface and security configurations. You will use the Development mode to expedite the setup process.

Development mode is intended for testing purposes only. It features insecure defaults, such as non-TLS communication and simplified security hardening. Never use this configuration in a production environment.

Task 1: Deploy Keycloak in Development Mode

Execute the following command to pull the Red Hat build of Keycloak 26.6 image and start the server.

1. Run the Keycloak container:

```
podman run --name mykeycloak \
  -p 127.0.0.1:8080:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin \
  -e KC_BOOTSTRAP_ADMIN_PASSWORD=change_me \
  registry.redhat.io/rhbk/keycloak-rhel9:26.6 \
```

```
  start-dev
```

2. **Explanation of parameters:**
   - `--name mykeycloak`: Assigns a human-readable name to your container.
   - `-p 127.0.0.1:8080:8080`: Maps the container's port 8080 to your local machine's port 8080.
   - `-e KC_BOOTSTRAP_ADMIN_USERNAME/PASSWORD`: Sets the initial administrative credentials.
   - `start-dev`: Instructs the server to run in development mode, enabling features like automatic TLS generation (self-signed) and relaxed security constraints.

Task 2: Verify the Installation

Once the container logs indicate that the server has started successfully, you must verify the accessibility of the Admin Console.

1. Open your web browser and navigate to: `http://localhost:8080`
2. Click the **Administration Console** link.
3. Log in using the credentials defined in Task 1:
   - **Username**: `admin`
   - **Password**: `change_me`

Troubleshooting

If you encounter issues during startup, check the following:

- **Port Conflicts**: If port 8080 is already in use on your host machine, you can change the host-side mapping. For example, use `-p 127.0.0.1:9090:8080` and access Keycloak at `http://localhost:9090`.
- **Registry Authentication**: If you receive an "access denied" error when pulling the image, ensure you are logged into the Red Hat Registry using `podman login registry.redhat.io`.
- **Logs**: If the container exits immediately, run `podman logs mykeycloak` to view the startup error logs.

Cleanup

Once you have completed your experimentation, stop and remove the container to free up system resources:

```
podman stop mykeycloak
podman rm mykeycloak
```

Authentication and Authorization

Authentication and Authorization

Authentication and Authorization are the core security pillars of the Red Hat build of Keycloak, designed to manage user identity and access control within applications.

- **Authentication:** Verifies user identity through configurable authentication flows, allowing for flexible login requirements and custom session management.
- **Authorization:** Enforces access control using Role-Based Access Control (RBAC) and permission-based models to ensure users only access authorized resources.
- **Customization:** Provides the capability to tailor authentication sessions to meet specific security policies and application needs.

User authentication flows

User authentication flows

Red Hat build of Keycloak provides a flexible and powerful mechanism for managing how users interact with the system through **Authentication Flows**. An authentication flow serves as a container for authentications, screens, and actions that occur during critical lifecycle events such as login, user registration, and credential recovery.

Understanding Authentication Flows

At its core, an authentication flow defines a sequence of steps that a user must complete to verify their identity or provide required information. Because modern security requirements vary across applications, Keycloak allows these flows to be highly customizable.

Keycloak organizes these interactions into specific execution types:

- **Login Flows:** Define the credential types required for access (e.g., password, One-Time Password (OTP), or WebAuthn).
- **Registration Flows:** Define the profile information a user must provide during sign-up and include security measures like reCAPTCHA to prevent bot-driven spam.
- **Credential Reset Flows:** Define the mandatory actions a user must perform to securely recover or reset a forgotten password.

Anatomy of a Flow

An authentication flow is structured as a tree of executions. Each execution is a specific action or validation step. You can configure these executions with different requirements:

- **REQUIRED:** The user must successfully complete this step to proceed.
- **ALTERNATIVE:** The user can choose between one or more steps (e.g., "Login with Password" OR "Login with OTP").

- **DISABLED:** The step is ignored during the flow execution.
- **CONDITIONAL:** The step is executed only if specific conditions are met.

Guided Exercise: Customizing a Login Flow

In this exercise, you will explore how to configure a simple authentication flow to enforce multi-factor authentication for a specific user group.

Prerequisites

- An active Red Hat build of Keycloak instance.
- Access to the Admin Console as a realm administrator.

Steps

1. **Navigate to Flows:** Log in to the Admin Console, select your Realm, and click **Authentication** in the left-hand navigation menu.
2. **View Existing Flows:** You will see a list of pre-configured flows. Select the **Browser** flow. This is the default flow used for web-based browser logins.
3. **Create a Copy:** It is best practice not to modify built-in flows directly. Click the **Duplicate** button next to the Browser flow and name it `Custom-Secure-Browser`.
4. **Add Execution:**
    - In your `Custom-Secure-Browser` flow, click the **Add step** button.
    - Select `OTP Form` from the list of available authenticators.
    - Change the requirement for `OTP Form` from `DISABLED` to `REQUIRED`.
5. **Bind the Flow:**
    - Navigate to the **Bindings** tab within the Authentication menu.
    - Locate the **Browser flow** dropdown and select your new `Custom-Secure-Browser` flow.
    - Click **Save**.

Verification

Open an Incognito window, navigate to your application, and attempt to log in. You will notice that after providing your standard username and password, the system now requires the additional OTP step, enforcing the new security policy defined in your flow.

Troubleshooting Tips

- **Flow Testing:** Always test new flows using an Incognito/Private browser window to ensure you do not lock yourself out of the Admin Console.
- **Requirement Conflicts:** If you set multiple authenticators as `REQUIRED`, ensure they are logically ordered. If a step fails, the entire authentication attempt will be rejected.
- **Execution Order:** You can drag and drop steps in the Admin Console to change their execution order. Always verify the sequence after making adjustments.

Role-based and permission-based authorization

Role-based and permission-based authorization

In the Red Hat build of Keycloak, managing access control is a fundamental security requirement. To simplify the complexity of managing permissions for thousands of individual users, Keycloak utilizes **Role-Based Access Control (RBAC)**.

Understanding Roles and Authorization

Authorization is the process of granting access to a user once they have been authenticated. Instead of assigning permissions to individual users—which is difficult to maintain and scale—you assign permissions to **roles**, and then map users to those roles.

Roles

Roles identify a category or "type" of user, such as `admin`, `manager`, `employee`, or `guest`. By defining roles, you create a logical abstraction that applications can use to check if a user is permitted to perform a specific action.

There are two primary scopes for roles in Keycloak: * **Realm Roles:** Global roles that apply to users across the entire realm. * **Client Roles:** Roles specific to a particular application (client). This allows you to have a `user` role for an internal inventory app that is distinct from a `user` role in a public-facing portal.

User Role Mapping

A **user role mapping** is the link between a specific user identity and a role. When a user authenticates, Keycloak includes these mappings in the security tokens (such as OIDC ID or Access tokens). Applications receive these tokens and inspect the roles to enforce access control decisions.

Permission-Based Authorization

While roles define **who** the user is, permission-based authorization defines **what** the user can do within an application.

In Keycloak, you can refine your security model by using **Groups**. Groups are collections of users that share common attributes or roles. Applying a role to a group automatically grants that role to every user within that group, streamlining the administration of large organizations.

Avoid assigning roles directly to individuals whenever possible. Use Groups to manage role assignments at scale. This minimizes administrative overhead and ensures consistent access policies across your user base.

Guided Exercise: Authorize Users with Red Hat build of Keycloak

In this exercise, you will create a role and map a user to it to test authorization.

Step 1: Create a Realm Role

1. Log in to the **Admin Console**.
2. Select your Realm from the dropdown menu.
3. In the left navigation menu, click **Realm roles**.
4. Click **Create role**.
5. Enter `manager` in the **Role name** field and click **Save**.

Step 2: Map a User to the Role

1. In the left navigation menu, click **Users**.
2. Search for the user you wish to authorize and click on their **Username**.
3. Navigate to the **Role mapping** tab.
4. Click **Assign role**.
5. Filter by "Realm roles," select the `manager` role, and click **Assign**.

Step 3: Verify the Mapping

You can now observe the user's effective permissions. When the user logs into an application configured with this Keycloak realm, the `manager` role will be included in their access token, signaling to the application that the user is authorized to perform manager-level tasks.

Lab: Authentication and Authorization

Goal: Configure a granular access control environment.

1. **Setup:** Create a new group named `FinanceTeam`.
2. **Action:** Create a client role named `report-viewer` within a specific Client (e.g., `finance-app`).
3. **Assignment:** Assign the `report-viewer` role to the `FinanceTeam` group.
4. **Validation:** Add a test user to the `FinanceTeam` group and verify that the user inherits the `report-viewer` role via the **Effective role** view in the User details page.

By completing these steps, you demonstrate how to decouple identity management (Users/Groups) from application access logic (Roles).

Customizing authentication sessions

Customizing Authentication Sessions

In Red Hat build of Keycloak, authentication sessions act as the "memory" of a login process, while user sessions represent the established trust once the user is successfully authenticated. Customizing these sessions is critical for balancing user convenience with security requirements.

Understanding Session Architecture

To effectively customize authentication, it is essential to distinguish between the two primary session states:

- **Authentication Sessions:** Track the transient state of an active login flow. This is where the server remembers which step of the authentication process the user is currently performing.
- **User Sessions:** Created upon successful authentication. This session tracks the active user and facilitates Single Sign-On (SSO), ensuring the user does not need to re-authenticate when accessing different applications.
- **Client Sessions:** A subset of the user session that tracks the user's state on a per-application (client) basis.

Configuring Session Lifespans

Keycloak provides granular control over how long sessions persist. You can adjust these settings to mitigate the risk of session hijacking or to meet specific compliance requirements regarding inactivity timeouts.

Session data is stored in the database by default and cached in the `sessions` and `clientSessions` caches for performance. Excessive session lifetimes can lead to increased memory pressure on your Infinispan clusters.

Guided Exercise: Adjusting Session Settings

In this exercise, you will modify the session timeouts to ensure a more secure user experience.

1. Log in to the **Admin Console**.
2. Select your desired **Realm** from the top-left dropdown.
3. In the left navigation menu, click **Realm settings**.
4. Navigate to the **Sessions** tab.
5. Configure the following settings:
    - **SSO Session Idle:** Sets the time after which a user session expires if there is no activity.
    - **SSO Session Max:** The maximum time a user session can remain active, regardless of activity.
    - **Client Session Idle:** The time a client session can remain idle before it is destroyed.
    - **Client Session Max:** The maximum time a client session can persist.
6. Click **Save**.

Customizing Authentication Sessions via Flows

Beyond session lifetimes, you can customize the authentication experience by modifying the authentication flows. By navigating to **Authentication** in the left menu, you can define how sessions are initiated and validated.

- **Browser Flows:** Customize how users interact with the login page.
- **Direct Grant Flows:** Customize how programmatic clients (non-browser) handle session creation.

Guided Exercise: Forcing Re-authentication

You can force a session to expire or require a re-authentication step by customizing the flow to include a "Conditional OTP" or "Re-authentication" execution.

1. Navigate to **Authentication** → **Flows**.
2. Select the **Browser** flow and click the **Actions** menu (three dots) → **Copy**.
3. Name the new flow `Custom-Secure-Browser-Flow`.
4. Add an execution of type **Re-authentication** to the flow.
5. Bind this new flow to the **Browser Flow** setting in **Realm Settings** → **Authentication**.

Lab: Managing Session Expiration

Scenario: Your security team requires that all user sessions be terminated after 30 minutes of inactivity and that no user session lasts longer than 8 hours, regardless of activity.

Steps: 1. Navigate to **Realm Settings** > **Sessions**. 2. Set **SSO Session Idle** to `30 minutes`. 3. Set **SSO Session Max** to `8 hours`. 4. Observe the impact: Users who step away from their desks for more than 30 minutes will be prompted to log in again upon returning to any protected application. 5. Verify the settings by logging in as a test user and checking the `sessions` count in the **Sessions** tab under the **Users** menu.

Guided Exercise: Authenticate Users with Red Hat build of Keycloak

Guided Exercise: Authenticate Users with Red Hat build of Keycloak

In this exercise, you will configure a basic authentication flow to verify user identity against the Red Hat build of Keycloak internal user database. We will define a realm, create a user, and test the authentication process using the Admin Console.

Prerequisites

- A running instance of Red Hat build of Keycloak 26.6.
- Access to the Keycloak Admin Console.

Step 1: Create a Realm

Realms are isolated environments that contain their own sets of users, credentials, roles, and groups.

1. Log in to the Admin Console.

2. In the top-left corner, click the dropdown menu and select **Create realm**.
3. Enter `training-realm` as the name.
4. Click **Create**.

Step 2: Create a User

Now, we will define a test user within the new realm.

1. In the left navigation menu, select **Users**.
2. Click **Add user**.
3. Enter the following details:
    - **Username:** `jdoe`
    - **Email:** `jdoe@example.com`
    - **First name:** `John`
    - **Last name:** `Doe`
4. Click **Create**.

Step 3: Set User Credentials

To allow `jdoe` to authenticate, you must define a temporary password.

1. Navigate to the **Credentials** tab within the user's profile.
2. Click **Set password**.
3. Enter a secure password (e.g., `Password123!`).
4. Toggle **Temporary** to **OFF** if you do not want the user to be forced to change the password upon their first login.
5. Click **Save** and confirm the change in the modal prompt.

Step 4: Verify Authentication

To verify that the authentication flow is working, we will use the built-in account console.

1. In the top-right corner of the Admin Console, click your administrator profile icon.
2. Select **Manage account**.
3. Log out of the admin session if prompted.
4. Attempt to log in using the credentials you just created:
    - **Username:** `jdoe`
    - **Password:** `Password123!`
5. If successful, you will be directed to the Keycloak Account Management console for `jdoe`.

Troubleshooting

- **Authentication Error:** Ensure the user is not locked and that the password was saved correctly. Check the "Attributes" tab to ensure the user is enabled.
- **Realm mismatch:** Ensure you are testing against `training-realm` and not the `master` realm.
- **Console access:** If the Account Console page does not load, verify that the `training-realm` is configured with the default browser flow in the **Authentication** tab.

Summary

You have successfully isolated a set of credentials within a dedicated realm and validated the identity verification process. This forms the foundation for more complex scenarios, such as OIDC integration or MFA enforcement, which you will explore in subsequent modules.

Guided Exercise: Authorize Users with Red Hat build of Keycloak

Guided Exercise: Authorize Users with Red Hat build of Keycloak

In this exercise, you will implement role-based access control (RBAC) to manage user permissions within the Red Hat build of Keycloak. You will define application-specific roles and map them to users to ensure secure authorization.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- An existing Realm and a test user account.
- Administrative access to the Keycloak Admin Console.

Step 1: Create a Realm Role

Realm roles are global roles that can be assigned to any user within the realm.

1. Log in to the **Keycloak Admin Console**.
2. Select your target realm from the top-left dropdown menu.
3. In the left navigation menu, click **Realm roles**.
4. Click **Create role**.
5. In the **Role name** field, enter `manager`.
6. Click **Save**.

Step 2: Create a User and Assign the Role

Now, you will associate the `manager` role with a specific user.

1. Navigate to **Users** in the left menu.

2. Select the user you wish to grant permissions to.
3. Click the **Role mapping** tab.
4. Click **Assign role**.
5. Filter for the `manager` role, select it, and click **Assign**.
    - The user now carries the `manager` authority claim within their security token.

Step 3: Verify Authorization via Token

When a user authenticates, Keycloak includes their roles in the ID or Access Token. You can inspect this token to confirm the authorization.

1. Use a client application (or the built-in Keycloak **Clients** evaluation tool) to perform a login flow for the user.
2. Observe the decoded JWT (JSON Web Token).
3. Locate the `realm_access` claim. It should appear similar to this:

```
"realm_access": {
  "roles": [
    "offline_access",
    "uma_authorization",
    "manager"
  ]
}
```

Step 4: Practical Activity - Resource Protection

To apply this authorization in a real-world scenario:

1. Navigate to **Clients** and select a client application.
2. Go to the **Authorization** tab (ensure Authorization is enabled for the client).
3. Create a new **Policy** based on the `manager` role:
    - Select **Role** as the policy type.
    - Add the `manager` role created in Step 1.
4. Create a **Permission** that links a Resource to the `manager` policy.
5. **Test:** Attempt to access the protected resource using a user account **without** the `manager` role. Verify that Keycloak (or the integrated application) returns a `403 Forbidden` response.

Troubleshooting Tips

- **Caching:** If you modify a user's role while they have an active session, the changes may not take effect until the user re-authenticates or the token expires.
- **Client Scopes:** Ensure that the `realm_access` claim is included in the client's token mapper settings if you are unable to see the role in the JWT.

Guided Exercise: Customize authentication with Red Hat build of Keycloak

Guided Exercise: Customize authentication with Red Hat build of Keycloak

In this exercise, you will customize the authentication flow in Red Hat build of Keycloak 26.6. By modifying authentication flows, you can move beyond simple username/password logins and enforce stronger security postures, such as requiring Multi-Factor Authentication (MFA) for specific client applications.

Prerequisites

- Access to a running Red Hat build of Keycloak 26.6 instance.
- An existing Realm and a test user account.
- Administrative access to the Admin Console.

Step 1: Accessing Authentication Flows

Authentication flows define the sequence of events that occur when a user attempts to log in.

1. Log in to the **Red Hat build of Keycloak Admin Console**.
2. Select your realm from the top-left dropdown menu.
3. In the left-hand navigation menu, click **Authentication**.
4. Review the list of available flows. You will see built-in flows such as `Browser`, `Direct Grant`, and `Registration`.

Step 2: Creating a Custom Flow

To customize authentication without breaking the default behavior, it is best practice to duplicate and modify a flow.

1. On the **Flows** tab, locate the **Browser** flow.
2. Click the **... (Actions)** menu on the right and select **Duplicate**.
3. Name the new flow `Custom-Browser-Flow` and provide a description.
4. Click **Save**.

Step 3: Adding an Authentication Execution

We will now add a requirement for OTP (One-Time Password) to the flow.

1. Click on the **Actions** menu for the `Custom-Browser-Flow` and click **Add execution**.
2. From the list of available providers, select **OTP Form**.
3. Click **Add**.
4. Locate the `OTP Form` row in your new flow. Change the requirement from `Disabled` to `Required`.

5. Move the `OTP Form` execution to the desired position in the flow (e.g., after the `Username Password Form`) using the drag-and-drop handles.

Step 4: Binding the Custom Flow

For the changes to take effect, you must bind the new flow to the Browser login process.

1. Navigate back to the **Authentication** page (Flows tab).
2. Click **Bindings** in the secondary navigation bar.
3. Locate the **Browser Flow** binding dropdown.
4. Select `Custom-Browser-Flow` from the list.
5. Click **Save**.

Step 5: Verification

1. Open a new Incognito/Private browser window.
2. Navigate to your application's login page or the Keycloak Account Console.
3. Enter your credentials.
4. Notice that after successful password authentication, Keycloak now redirects you to the OTP setup or verification screen, confirming that your custom authentication flow is active.

Troubleshooting Tips

- **Reverting Changes:** If you lock yourself out of the Admin Console, use the `kcadm.sh` CLI tool to revert the flow binding:

```
./kcadm.sh update realms/my-realm -s browserFlow=browser
```

- **Logging:** Ensure the log level for `org.keycloak.authentication` is set to `DEBUG` in your configuration if flows are failing to execute as expected. You can update this via the build options in the Keycloak configuration file.

Always test custom authentication flows in a staging environment before applying them to production realms to prevent user lockout.

Lab: Authentication and Authorization

Lab: Authentication and Authorization

In this lab, you will implement secure access control by configuring authentication flows and enforcing authorization policies within the Red Hat build of Keycloak. You will transition from basic user authentication to granular permission-based access control.

Prerequisites

- A running instance of Red Hat build of Keycloak in development mode.
- Access to the Admin Console with administrative credentials.
- A pre-configured realm (e.g., `lab-realm`).

Lab Scenario

Your organization requires a secure portal for internal applications. You must: 1. Configure a user authentication flow. 2. Define roles and map them to users. 3. Enforce authorization based on user roles.

Step 1: Configure Authentication Flows

In this task, you will create a custom authentication flow that requires a password update upon the first login.

```
# 1. Navigate to 'Authentication' in the Admin Console.
# 2. Click 'Create flow' and name it 'Custom-Auth-Flow'.
# 3. Add 'Password Reset' execution to the flow.
# 4. Set the flow to 'Required' for the Browser flow.
```

Step 2: Define Roles and Permissions

You will now define role-based access control (RBAC).

1. Define two roles: `manager` and `employee`.
2. Create a user `jsmith` and assign the `employee` role.
3. Create a user `admin_user` and assign the `manager` role.

Use the Admin CLI (`kcadm.sh`) to assign roles efficiently.

```
# Example: Mapping a role to a user via CLI
./kcadm.sh add-roles --uusername jsmith --rolename employee -r lab-
realm
```

Step 3: Implement Authorization Policies

In this section, you will enforce that only users with the `manager` role can access a specific client resource.

1. Select your client under the **Clients** menu.
2. Enable **Authorization Enabled** in the Client settings.
3. Navigate to the **Authorization** tab.
4. Create a new **Role-Based Policy** named `manager-policy` and select the `manager` role.

5. Create a **Permission** that links the `manager-policy` to your specific resource.

Step 4: Verification

Verify the configuration by attempting to access protected resources.

1. Log into the Account Console as `jsmith`. Verify the user has access to basic profile settings but is restricted from management endpoints.
2. Log into the Account Console as `admin_user`. Confirm that the administrative features defined in your authorization policy are accessible.

Troubleshooting

- **User cannot log in:** Check the Authentication flow logs in the server console to ensure the execution path is correctly configured.
- **Access Denied errors:** Verify that the "Authorization Enabled" toggle is active on the client and that the user is correctly mapped to the required role in the target realm.
- **Token issues:** Use the JWT Debugger to inspect the access token generated for `admin_user` and ensure the roles are being correctly embedded in the `realm_access` claim.

Quiz: Check Your Understanding

1. Why is it recommended to use Role-Based Access Control (RBAC) instead of assigning permissions directly to users?
2. Which standard (OIDC or SAML) is primarily used by the Red Hat build of Keycloak to transport authorization claims in a token?
3. How does "Credential Isolation" protect user security during the authentication flow?

Identity Brokering and User Federation

Identity Brokering and User Federation

Identity Brokering and User Federation in Red Hat build of Keycloak allow organizations to centralize authentication by integrating with external identity sources.

- **Identity Brokering:** Acts as an intermediary service, enabling users to authenticate against external identity providers (IdPs), such as social networks, business partners, or cloud-based services, using standard protocols like SAML v2.0.
- **User Federation:** Facilitates the integration of existing user directories, such as LDAP or Active Directory, allowing Keycloak to manage and authenticate users stored in external repositories.
- **Identity Provider Mappers:** Used during federation to map incoming tokens and assertions to local user and session attributes, ensuring identity information is accurately propagated to requesting clients.
- **User Management:** Enables users to link their local accounts with multiple external identities or automatically create accounts based on information provided by the integrated IdP.

LDAP and Active Directory integration

LDAP and Active Directory Integration

Red Hat build of Keycloak provides robust support for integrating existing directory services, allowing organizations to centralize identity management. By leveraging LDAP (Lightweight Directory Access Protocol) and Microsoft Active Directory (MSAD), you can delegate user authentication and attribute synchronization to your existing enterprise infrastructure.

Overview of User Federation

User federation allows Keycloak to act as a bridge between your applications and an external user store. Instead of migrating all users into the Keycloak database, Keycloak queries your LDAP or Active Directory server in real-time or via periodic synchronization.

Key benefits include: * **Single Source of Truth:** Manage user lifecycles in your existing directory. * **Seamless SSO:** Users utilize their corporate credentials to access applications protected by Keycloak. * **Flexible Mapping:** Map LDAP attributes (like `department` or `employeeID`) to Keycloak user attributes or tokens.

Configuring LDAP Integration

To connect your directory service, you must define an LDAP provider within a specific Realm.

1. In the Admin Console, navigate to **User Federation**.
2. Click **Add provider** and select **ldap**.
3. Configure the connection settings:

- **Connection URL:** The LDAP/LDAPS URL (e.g., `ldap://ldap.example.com:389`).
- **Users DN:** The base DN where your users reside (e.g., `ou=users,dc=example,dc=com`).
- **Bind DN/Password:** Credentials used by Keycloak to query the directory.

Ensure you set the **LDAP bind credential** correctly in the LDAP settings. This credential must have read permissions for the specified User DN subtree to allow Keycloak to authenticate and fetch user details.

Integrating Microsoft Active Directory

Active Directory requires specific handling for user account control and state management. Keycloak includes dedicated mappers to ensure compatibility with AD-specific attributes.

Adding an MS Active Directory Mapper

When integrating with MSAD, you often need to map the `userAccountControl` attribute to handle disabled accounts correctly. You can automate this process using the Admin CLI (`kcadm.sh`).

Procedure

1. Identify the `parentId` of your existing LDAP provider by listing current components:

```
kcadm.sh get components -r <realm-name>
```

2. Create the MSAD mapper by executing the `create` command:

```
$ kcadm.sh create components -r demorealm \
  -s name=msad-user-account-control-mapper \
  -s providerId=msad-user-account-control-mapper \
  -s providerType=org.keycloak.storage.ldap.mappers.LDAPStorageMapper \
  -s parentId=<your-ldap-provider-id>
```

Guided Exercise: Federating Users with LDAP

In this exercise, you will establish a connection to an LDAP server and verify connectivity.

Step 1: Define the LDAP Provider Navigate to the Admin Console, select your realm, and add the LDAP provider using the connection details provided by your infrastructure team.

Step 2: Test the Connection Use the "Test connection" and "Test authentication" buttons within the LDAP provider configuration page to verify that the credentials and DN structure are valid.

Step 3: Synchronize Users Use the **Synchronize all users** action to import existing directory users into the Keycloak user cache, ensuring they are searchable for role assignments.

Lab: Authentication and Authorization with LDAP

1. **Task 1:** Configure an LDAP connection in the `demorealm`.

2. **Task 2:** Add an MS Active Directory mapper to handle user status synchronization.
3. **Task 3:** Enable "Import Users" to allow Keycloak to mirror the LDAP user profile in its internal database.
4. **Task 4:** Attempt to log in via the Account Console using a credential existing only in the LDAP server. Verify that the user record appears in the Keycloak Admin Console after the first successful login.

Delegating authentication to third-party identity providers

Delegating Authentication to Third-Party Identity Providers

Overview

Delegating authentication allows your applications to offload the responsibility of verifying user credentials to an external system. Within the Red Hat build of Keycloak, this is facilitated through the **Identity Broker** component.

By acting as an intermediary, Red Hat build of Keycloak enables users to sign in using accounts from external identity providers (IDPs), such as Google, GitHub, or another SAML-based enterprise system, while maintaining a unified security policy for your internal applications.

How Identity Delegation Works

When a user attempts to access your application, Red Hat build of Keycloak intercepts the request and redirects the user to the configured external IDP. Once the external IDP authenticates the user, it sends an assertion or token back to Red Hat build of Keycloak.

Keycloak then validates this response and issues its own internal token to the client application. This process ensures that: * **Protocol Abstraction:** The client application only needs to understand the Red Hat build of Keycloak protocol (OIDC/SAML), regardless of how the user initially authenticated. * **Centralized Management:** You manage your authentication policies in one place, even if users come from diverse sources. * **Seamless Experience:** Users can access your services using existing credentials without creating new, siloed accounts.

Identity Provider Mappers

A critical aspect of delegating authentication is ensuring that the data provided by the external IDP is correctly ingested by your local environment. **Identity Provider Mappers** allow you to transform and map incoming tokens or assertions into user attributes or session information. This is essential for: * Synchronizing user profile data (e.g., email, first name) from the external provider to the local user record. * Mapping external roles to local groups or roles within your realm. * Propagating session data to ensure the client application has the necessary context to authorize the user.

Guided Exercise: Integrating an External OIDC Identity Provider

In this exercise, you will configure an external OpenID Connect (OIDC) provider as an Identity Broker.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- A registered application (client) on the target external OIDC provider (e.g., Google or another Keycloak instance).

Steps

1. **Access the Admin Console**: Log in to the Red Hat build of Keycloak Admin Console.
2. **Select Realm**: Select the realm where you want to configure the brokering.
3. **Navigate to Identity Providers**:
   - In the left-hand menu, click **Identity providers**.
   - Click the **Add provider** dropdown and select **OpenID Connect v1.0**.
4. **Configure Provider Details**:
   - **Alias**: Enter a unique name for this provider (e.g., `external-oidc`).
   - **Client ID & Client Secret**: Enter the credentials provided by the external OIDC provider.
   - **Discovery Endpoint**: Enter the URL of the external OIDC provider's `.well-known/openid-configuration`.
5. **Save and Test**:
   - Click **Save**.
   - Use the **Redirect URI** provided in the settings to ensure your external IDP has the correct callback URL configured.
   - Open a new browser session and navigate to your application. You should now see the external provider listed on your login page.

Troubleshooting Tips

- **Invalid Redirect URI**: Ensure the `Redirect URI` in the Keycloak Identity Provider configuration matches exactly what is registered in the external IDP's dashboard.
- **Token Mapping Issues**: If user attributes are not appearing after login, check the **Mappers** tab in the Identity Provider configuration. Ensure that the mapper type matches the data structure expected from the provider.
- **Clock Skew**: If authentication fails with "invalid token," ensure the system time on your Keycloak server is synchronized via NTP, as clock skew can cause token validation to fail.

Social login configuration

Social Login Configuration

In modern identity management, users increasingly prefer the convenience of using existing identities to access new applications. Red Hat build of Keycloak simplifies this by acting as an Identity Broker, allowing you to delegate authentication to trusted third-party social networks.

Understanding Social Identity Providers

Social identity providers enable your users to authenticate using their existing accounts from platforms like Google, GitHub, Facebook, and Microsoft. By configuring these providers, you reduce friction during the user onboarding process, as users do not need to create and remember yet another set of credentials.

Keycloak handles the complex OAuth 2.0 or OpenID Connect handshake behind the scenes, ensuring that the identity exchange is secure and that user attributes are mapped correctly into your realm.

Supported Providers

Red Hat build of Keycloak provides native support for a wide array of social platforms, including: * **Development & Collaboration:** GitHub, GitLab, Bitbucket, Stack Overflow * **Social & Professional:** Facebook, LinkedIn, Twitter, Instagram * **Enterprise & Cloud:** Google, Microsoft, OpenShift v4, PayPal

Guided Exercise: Integrating Social Login

This exercise walks you through the steps required to configure a social identity provider in your Keycloak realm.

Prerequisites

- Access to the Keycloak Administration Console.
- An existing client application (e.g., a "Client ID" and "Client Secret") registered with the provider you choose (e.g., Google or GitHub).

Procedure

1. **Access Identity Providers:** Log in to the Keycloak Admin Console. In the left-hand navigation menu, click **Identity Providers**.
2. **Add a Provider:** Locate the **Add provider** dropdown menu. This list contains all supported social networks and standard protocols. Select your desired provider (e.g., **Facebook** or **GitHub**).
3. **Configure Provider Details:** Once selected, the configuration screen appears. You must provide the following:
    - **Client ID:** The identifier provided by the social network when you registered your application.
    - **Client Secret:** The secret key associated with your application on the social network's developer dashboard.

4. **Save and Enable:** Click **Add** or **Save**. Ensure the **Enabled** toggle is set to `On`.
5. **Verify the Flow:** Navigate to your application's login page. You will see a button corresponding to the social provider you just configured. When a user clicks this button, they will be redirected to the social network to authenticate. Upon successful login, they are redirected back to Keycloak, where a local user account is automatically linked or created based on your mapping configuration.

Best Practices for Social Brokering

- **Attribute Mapping:** Use the **Mappers** tab within your Identity Provider configuration to sync information like email addresses, first names, and last names from the social provider to the Keycloak user profile.
- **First Broker Login Flow:** Keycloak uses a default "First Broker Login" flow. You can customize this flow if you need to perform additional actions, such as forcing users to update their profile or accept terms of service upon their first social login.
- **Security:** Always use HTTPS for your Keycloak deployment. Since you are handling tokens from third-party providers, ensuring the communication channel between the user, the social provider, and Keycloak is encrypted is critical.

Guided Exercise: Federating Users with Red Hat build of Keycloak

Guided Exercise: Federating Users with Red Hat build of Keycloak

In this exercise, you will connect your Red Hat build of Keycloak instance to an external user store. User federation allows you to offload user authentication and management to existing enterprise directories, such as LDAP or Active Directory, without migrating your user data into the Keycloak database.

Prerequisites

- A running instance of Red Hat build of Keycloak (version 26.6).
- Administrative access to the Keycloak Admin Console.
- Access to an accessible LDAP or Active Directory server (or a configured test instance).

Task 1: Adding a User Federation Provider

You will configure Keycloak to communicate with an LDAP server to verify user credentials.

1. Log in to the **Keycloak Admin Console**.
2. Select your target **Realm** from the dropdown menu in the top-left corner.
3. In the left-hand navigation menu, click **User federation**.
4. Click the **Add user federation provider** button.
5. Select **ldap** from the provider list.

6. Configure the following fields in the **General Options** section:
    - **Console Display Name**: `MyEnterpriseLDAP`
    - **Import Users**: Set to `ON` (This creates a local copy of the user in Keycloak for better performance).
7. Configure the **LDAP Connection Data**:
    - **Connection URL**: Enter the URL of your LDAP server (e.g., `ldap://ldap.example.com:389`).
    - **Users DN**: Enter the base DN for your users (e.g., `ou=users,dc=example,dc=com`).
    - **Bind DN** and **Bind Credential**: Provide the credentials for the service account Keycloak will use to read the directory.
8. Click **Save**.

Task 2: Verifying Connection and Mapping

Once the provider is saved, you must ensure that Keycloak can communicate with the server and map LDAP attributes to Keycloak user attributes.

1. Click the **Test connection** button to ensure the URL and bind credentials are correct.
2. Click the **Test authentication** button to verify that the service account can authenticate against the LDAP server.
3. Navigate to the **Mappers** tab.
4. Click **Add mapper** or use the **Create mappers from template** button to automatically map standard LDAP attributes (such as `mail`, `givenName`, and `sn`) to Keycloak user attributes.

Task 3: Testing User Login

Now, you will verify that a user from your LDAP directory can authenticate through Keycloak.

1. Open an incognito browser window and navigate to your application's login page or the Keycloak Account Console.
2. Attempt to log in using the credentials of an existing user present in your LDAP directory.
3. Upon successful login, navigate back to the **Keycloak Admin Console**.
4. Click **Users** in the left navigation menu.
5. Search for the user you just logged in with.
6. Verify that the user now appears in the list. Note the "Federation Link" attribute, which indicates the user is sourced from your LDAP provider.

Troubleshooting Tips

- **Logs**: If the connection fails, check the server logs. In the Keycloak terminal, look for `LDAPException` or authentication failure messages.
- **Firewall**: Ensure the server running Red Hat build of Keycloak can reach the LDAP/AD server on the specified port (usually 389 for LDAP or 636 for LDAPS).
- **Attribute Mapping**: If user profiles appear empty, re-check the **Mappers** tab to ensure the

LDAP attribute names match your directory schema exactly.

When using the "Import Users" feature, Keycloak caches the user data. If you update a user's information in LDAP, you may need to trigger a manual sync or wait for the configured sync interval in the federation provider settings for those changes to reflect in Keycloak.

Guided Exercise: Integrating Social Login with Red Hat build of Keycloak

Guided Exercise: Integrating Social Login with Red Hat build of Keycloak

In this exercise, you will configure Red Hat build of Keycloak to allow users to authenticate using an external social identity provider (IdP). By offloading authentication to providers like Google or GitHub, you reduce the burden of password management and improve the user experience.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- An account with an external Identity Provider (e.g., Google Cloud Console or GitHub Developer Settings) to obtain `Client ID` and `Client Secret`.
- Access to the Keycloak Admin Console.

Step 1: Obtain Credentials from the Identity Provider

Before configuring Keycloak, you must register your application with the social provider.

1. Log in to your social provider's developer console (e.g., https://console.cloud.google.com/ for Google).
2. Create a new OAuth 2.0 Client ID.
3. Set the **Authorized redirect URI** to the value provided by Keycloak, which typically follows this pattern: `https://<KEYCLOAK_HOST>/realms/<REALM_NAME>/broker/google/endpoint`
4. Copy the generated **Client ID** and **Client Secret**.

Step 2: Configure the Identity Provider in Keycloak

Now, enable the integration within the Keycloak Admin Console.

1. Log in to the Admin Console.
2. Select the **Realm** you want to configure from the top-left dropdown.
3. In the left-hand menu, click **Identity providers**.
4. Click the **Add provider** dropdown and select your desired provider (e.g., **Google**).
5. Enter the **Client ID** and **Client Secret** you obtained in Step 1.
6. Note the **Redirect URI** provided on this page to ensure it matches the configuration in your

developer console.

7. Click **Add**.

Step 3: Configure User Mapping (Optional)

To ensure that information from the social provider (like email or name) is correctly stored in Keycloak, configure mappers.

1. Within the Identity Provider configuration screen, click the **Mappers** tab.
2. Click **Add mapper**.
3. Select the attributes you wish to map (e.g., `Email`, `First Name`, `Last Name`).
4. Click **Save**.

Step 4: Verify the Integration

Test the configuration to ensure the authentication flow works as expected.

1. Open your application's login page or the Keycloak Account Console.
2. Observe that a new button (e.g., "Sign in with Google") appears on the login screen.
3. Click the button and authenticate with your social account credentials.
4. If this is your first time logging in via this provider, Keycloak may prompt you to link the social account to a local user account or register a new one.

Troubleshooting Tips

- **Invalid Redirect URI:** Ensure the URI in the social provider's console exactly matches the one displayed in the Keycloak Identity Provider configuration.
- **HTTPS Requirements:** Many social providers require your Keycloak instance to be served over HTTPS. If you are in development mode on `localhost`, ensure your browser allows the callback.
- **Permissions:** Ensure the API/Scope permissions for the Client ID in the social provider console include `profile` and `email` access.

Conclusion

You have successfully configured Red Hat build of Keycloak to delegate authentication to a social identity provider. Users can now authenticate using their existing social accounts, while you maintain centralized control over their access and roles within your Keycloak realms.

Lab: Identity Brokering and User Federation

Lab: Identity Brokering and User Federation

In this lab, you will configure Red Hat build of Keycloak to act as an Identity Broker. You will integrate an external Identity Provider (IdP) to allow users to authenticate using credentials from an external source, and map the incoming identity information to your local realm.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- Administrative access to the Admin Console.
- Access to an external Identity Provider (e.g., a Google developer account or a SAML-based IdP).

Objectives

- Configure a new Identity Provider in the Keycloak Admin Console.
- Set up Identity Provider Mappers to sync user attributes.
- Test the authentication flow via the brokered Identity Provider.

Exercise 1: Configuring an Identity Provider

In this task, you will create a brokering relationship with an external provider.

1. Log in to the **Keycloak Admin Console**.
2. In the left-hand menu, click **Identity providers**.
3. Select an provider from the **Add provider** dropdown (e.g., `Google` or `SAML v2.0`).
4. Configure the required fields based on your external provider's documentation:
    - **Redirect URI**: Copy this value to your external provider's configuration console as the "Authorized Redirect URI."
    - **Client ID/Client Secret**: Enter the credentials obtained from the external provider.
5. Click **Save**.

Exercise 2: Implementing Identity Provider Mappers

Identity provider mappers allow you to propagate information from the external IdP to your local user sessions.

1. Within your newly created Identity Provider configuration, navigate to the **Mappers** tab.
2. Click **Add mapper**.
3. Configure the mapper to extract identity information:
    - **Name**: `Sync Email`
    - **Mapper type**: `Attribute Importer`
    - **Identity Provider Attribute**: `email` (or the specific attribute name provided by the IdP).
    - **User Attribute**: `email`
4. Click **Save**.

When using Identity Brokering APIs V2, remember that Keycloak can store tokens issued by the external IdP. If your application needs to call the external provider's API on behalf of the user, ensure "Store tokens" is enabled in the provider settings.

Exercise 3: Verifying the Integration

Now, test that users can log in via the external broker.

1. Open your application or the Keycloak Account Console in an incognito browser window.
2. Navigate to the login page.
3. You should now see the external Identity Provider button displayed.
4. Click the button and authenticate using the external provider's credentials.
5. Upon successful authentication, check the **Users** section in the Admin Console. You should see a new user created (or linked) with the attributes mapped from the Identity Provider.

Troubleshooting

- **Redirect URI Mismatch**: Ensure the Redirect URI in Keycloak matches exactly what is configured in the external IdP portal.
- **Protocol Errors**: If using SAML, verify that the Identity Provider metadata URL or XML file is reachable by your Keycloak instance.
- **Logging**: Monitor the Keycloak server logs for detailed authentication handshake errors:

```
# View server logs to debug authentication flows
tail -f /opt/keycloak/data/log/keycloak.log
```

Summary

You have successfully configured Red Hat build of Keycloak as an Identity Broker, mapped external attributes to the local user database, and verified the authentication flow. This architecture allows for a centralized user management strategy across disparate security domains.

Red Hat build of Keycloak on Red Hat OpenShift

Red Hat build of Keycloak on Red Hat OpenShift

Red Hat build of Keycloak (RHBK) is designed to run efficiently in containerized environments, providing robust identity and access management for applications deployed on Red Hat OpenShift.

Key aspects of running RHBK on OpenShift include:

- **Platform Support:** Deployment follows established Red Hat Middleware guidelines and aligns with the Red Hat OpenShift Container Platform Life Cycle Policy.
- **Version Compatibility:** Support is validated across multiple OpenShift releases (e.g., 4.12 through 4.21, depending on the RHBK version) ensuring a stable infrastructure foundation.
- **Architecture Flexibility:** RHBK supports diverse chipset architectures, including x86_64, s390x, ppc64le, and Aarch64, providing deployment versatility.
- **Standardized Runtime:** The solution utilizes the UBI9-based OpenJDK 17 container image, ensuring consistency, security, and performance within the OpenShift cluster.
- **Lifecycle Management:** Configuration and operational management leverage native OpenShift capabilities to simplify scaling, deployment, and maintenance of identity services.

Deploying Keycloak in containerized environments

Deploying Red Hat build of Keycloak in Containerized Environments

In modern architecture, the Red Hat build of Keycloak is designed to run as an immutable container. Containerization allows for consistent deployment across environments—from local development to large-scale production clusters.

Containerization Principles

When deploying Red Hat build of Keycloak in a containerized environment, adhere to these core principles:

- **Immutability:** Treat containers as ephemeral. Rather than modifying a running container, rebuild and redeploy the image with updated configurations or versions.
- **Performance Optimization:** Use optimized images to ensure fast startup times, which is critical for auto-scaling and frequent re-provisioning.
- **Security Isolation:** Leverage container-native security to protect sensitive credentials and network traffic.

Running the Standard Container

For environments where rapid startup and pre-optimized image layers are not the primary

constraint, you can deploy the standard Red Hat build of Keycloak image.

```
podman run --name mykeycloak -p 127.0.0.1:8080:8080 \
  -e KEYCLOAK_ADMIN=admin \
  -e KEYCLOAK_ADMIN_PASSWORD=change_me \
  quay.io/keycloak/keycloak:latest \
  start-dev
```

Initial Admin Credentials

By default, the Red Hat build of Keycloak restricts the creation of the initial administrator to local network connections. Because containers represent a network boundary, you must explicitly pass administrative credentials via environment variables during the initial container startup.

- `KEYCLOAK_ADMIN`: Sets the username for the initial admin account.
- `KEYCLOAK_ADMIN_PASSWORD`: Sets the password for the initial admin account.

Always manage these credentials via secure mechanisms, such as OpenShift Secrets or Kubernetes Secrets, rather than hardcoding them in command-line scripts or container manifests.

Guided Exercise: Deploying a Containerized Keycloak

In this exercise, you will deploy a standard Red Hat build of Keycloak container instance.

Prerequisites

- A container runtime installed (e.g., Podman or Docker).
- Network access to the Red Hat Quay registry.

Steps

1. **Pull the Image:** Ensure you have the latest image from the Red Hat registry.

   ```
   podman pull quay.io/keycloak/keycloak:latest
   ```

2. **Run the Container:** Execute the container using environment variables to define your admin credentials.

   ```
   podman run --name keycloak-dev -p 8080:8080 \
     -e KEYCLOAK_ADMIN=admin \
     -e KEYCLOAK_ADMIN_PASSWORD=adminpassword \
     quay.io/keycloak/keycloak:latest \
     start-dev
   ```

3. **Verify Deployment:** Open your web browser and navigate to `http://localhost:8080`. Log in

using the credentials defined in the environment variables to access the Admin Console.

Production Considerations

While running a container with `start-dev` is suitable for local exploration, transitioning to a production-ready containerized environment involves additional layers of configuration:

- **Database Connectivity:** Configure external JDBC-compliant databases (PostgreSQL, MariaDB, etc.) rather than using the default embedded H2 database.
- **TLS/SSL Termination:** Ensure secure communication by offloading TLS at the Load Balancer or Ingress controller level.
- **Configuration Management:** Use environment variables, CLI options, or mounted configuration files to manage Keycloak build-time and run-time parameters consistently across your CI/CD pipeline.

Configuration and management on OpenShift

Configuration and Management on Red Hat build of Keycloak on OpenShift

When deploying the Red Hat build of Keycloak on OpenShift, configuration and management move away from traditional file-based editing to a declarative, container-native approach. This section covers the essential mechanisms for managing your Keycloak instance within an OpenShift environment.

Operational Considerations for OpenShift

Unlike standalone deployments, Red Hat build of Keycloak on OpenShift relies on the OpenShift Operator for lifecycle management. Key configurations—such as environment variables, secrets, and truststores—are managed via custom resources (CRs) that the Operator reconciles.

Truststore Configuration

To secure communication between Keycloak and external services (like an OpenShift API server for identity brokering), you must manage a custom Truststore.

The certificate of the OpenShift 4 instance must be stored in the Red Hat build of Keycloak Truststore. Ensure your Keycloak server is explicitly configured to utilize this truststore.

To integrate an OpenShift v4 instance as an Identity Provider, follow these configuration steps:

1. **Retrieve the API URL**: Run the following command to identify your cluster's API endpoint:

```
oc cluster-info
```

2. **Identify the URL**: Locate the output formatted as: `Kubernetes master is running at https://api.<your-openshift-domain>:6443`.

3. **Configure the Identity Provider**:
    - Navigate to the **Admin Console**.
    - Select **Identity Providers** in the left-hand menu.
    - Select **OpenShift v4** from the **Add provider** dropdown.
    - Input the **Client ID**, **Client Secret**, and the **Base URL** (the API URL obtained in step 2).

High Availability (HA) and Multi-Site Deployments

For production environments requiring high availability, Red Hat provides certified integration components. When configuring your OpenShift deployment, ensure you align with these supported versions:

Component	Supported Versions (Minimum)	Note
OpenShift Container Platform	4.17+	Required for multi-AZ or on-prem deployments.
Red Hat Data Grid	8.5.x	Required for multi-cluster ROSA deployments.
AWS Aurora PostgreSQL	17.x	Required for multi-cluster ROSA deployments.

The image-based configuration provided by the Red Hat build of Keycloak is strictly designed for OpenShift environments. Using these specific configurations on generic Kubernetes distributions is not supported.

Guided Exercise: Configuring Identity Brokering with OpenShift

This exercise demonstrates how to link your OpenShift 4 instance as an identity provider to your Keycloak realm.

Prerequisites: * A running Red Hat build of Keycloak instance on OpenShift. * The OpenShift cluster certificate imported into the Keycloak truststore.

1. **Step 1: Accessing the Admin Console** Login to the Admin Console using your administrator credentials.
2. **Step 2: Adding the Provider** In the navigation menu, click **Identity Providers**. From the **Add provider** list, select **Openshift v4**.
3. **Step 3: Defining Parameters**
    - **Client ID/Secret**: Enter the credentials generated for your Keycloak client within the OpenShift project.
    - **Base URL**: Enter the API URL retrieved via `oc cluster-info`.
    - **Redirect URI**: Copy the generated Redirect URI provided in the console; you will need this to configure the OAuth client in the OpenShift console.

4. **Step 4: Finalizing** Save the configuration. Users will now see the "Log in with OpenShift" option on the login page.

Troubleshooting Tips

- **Truststore Issues**: If authentication fails with a `PKIX path building failed` error, verify that your OpenShift API server certificate has been correctly injected into the Keycloak Truststore secret and referenced in the `Keycloak` Custom Resource.
- **Compatibility**: Always verify that your OpenShift version matches the certified version (4.17+) to ensure full support for the Keycloak Operator features.

Guided Exercise: Install Red Hat build of Keycloak on Red Hat OpenShift

Guided Exercise: Install Red Hat build of Keycloak on Red Hat OpenShift

In this exercise, you will deploy the Red Hat build of Keycloak on an existing Red Hat OpenShift cluster. We will utilize the Keycloak Operator, which is the recommended approach for managing Keycloak instances in a containerized environment to ensure automated lifecycle management.

Prerequisites

- Access to a Red Hat OpenShift cluster with `cluster-admin` or appropriate namespace permissions.
- The `oc` CLI tool authenticated to your cluster.
- Access to the Red Hat OperatorHub.

Step 1: Create a Namespace

To maintain a clean environment, we will isolate the Keycloak deployment in its own project.

```
oc new-project keycloak-system
```

Step 2: Install the Red Hat build of Keycloak Operator

The Operator manages the installation, upgrades, and configuration of Keycloak instances.

1. Navigate to the OpenShift Web Console.
2. Go to **Operators** > **OperatorHub**.
3. Search for **"Red Hat build of Keycloak"**.
4. Select the operator and click **Install**.
5. Ensure the "Installation mode" is set to "A specific namespace on the cluster" and select `keycloak-system`.
6. Click **Install**.

Step 3: Define the Keycloak Custom Resource

Once the operator status is "Succeeded," we must define the `Keycloak` Custom Resource (CR) to initiate the deployment.

Create a file named `keycloak-cr.yaml`:

```
apiVersion: k8s.keycloak.org/v2alpha1
kind: Keycloak
metadata:
  name: keycloak
  namespace: keycloak-system
spec:
  instances: 1
  image: registry.access.redhat.com/rhbk/keycloak-26-rhel9:latest
  db:
    vendor: postgres
    host: keycloak-db-service
  hostname:
    hostname: keycloak.example.com
  tls:
    certificateSecret:
      secretName: keycloak-tls-secret
```

NOTE: Ensure you have a corresponding PostgreSQL database service and a TLS secret configured before applying this manifest, as Keycloak requires a relational database for production-grade configurations.

Step 4: Apply the Configuration

Deploy the instance by applying the YAML file via the CLI:

```
oc apply -f keycloak-cr.yaml
```

Step 5: Verify the Installation

Monitor the progress of your deployment using the following command:

```
oc get keycloak/keycloak -n keycloak-system -w
```

Wait until the `STATUS` field transitions to `Running` and the `READY` field indicates the pods are successfully initialized.

Troubleshooting

- **Check Operator Logs:** If the installation stalls, use `oc logs deployment/keycloak-operator -n keycloak-system` to identify configuration errors.

- **Pod Resource Constraints:** If the Keycloak pod enters a `CrashLoopBackOff`, verify your cluster has sufficient CPU and memory resources to pull the image and start the JVM.
- **Database Connectivity:** Ensure that the database credentials and hostnames provided in the `db` spec are reachable from within the `keycloak-system` namespace.

Guided Exercise: Configure Red Hat build of Keycloak on Red Hat OpenShift

Guided Exercise: Configure Red Hat build of Keycloak on Red Hat OpenShift

This exercise guides you through the post-deployment configuration of the Red Hat build of Keycloak on an OpenShift cluster. After deploying the operator and the Keycloak custom resource, you must ensure the environment is optimized for secure production-like operations.

Prerequisites

- A running Red Hat OpenShift cluster.
- Red Hat build of Keycloak Operator installed via OperatorHub.
- An existing `Keycloak` custom resource (CR) instance deployed in your project.

Task 1: Verify the Keycloak Custom Resource

Before modifying configurations, ensure your Keycloak instance is healthy and the status is ready.

1. Log in to your OpenShift cluster via the CLI:

```
oc login -u <username>
oc project <your-keycloak-project>
```

2. Verify the status of the Keycloak instance:

```
oc get keycloak <instance-name> -o yaml
```

3. Ensure the `status.ready` field is set to `true`.

Task 2: Configure TLS and Ingress

In OpenShift, the Keycloak route is managed by the Ingress Controller. To secure the communication, you should ensure proper TLS termination.

1. Retrieve the default route created by the Keycloak operator:

```
oc get routes
```

2. If you are using custom certificates, patch the route to point to your Secret containing the certificate and key:

```
oc patch route <route-name> --type=merge -p
'{"spec":{"tls":{"termination":"reencrypt","certificate":"<your-
cert>","key":"<your-key>"}}}'
```

Task 3: Update Environment Variables

You can update the configuration of the running Keycloak instance by modifying the `Keycloak` custom resource. This triggers a rolling update of the pods.

1. Edit the Keycloak resource to include custom build options or environment settings (e.g., database connection pooling or logging levels):

```
oc edit keycloak <instance-name>
```

2. Add or modify the `spec.additionalOptions` section:

```
spec:
  additionalOptions:
    - name: http-relative-path
      value: /auth
    - name: log-level
      value: INFO
```

3. Save and exit the editor. The operator will automatically reconcile the pods to reflect these changes.

Task 4: Validate Configuration Changes

After the pods restart, verify that the new configurations have been applied.

1. Monitor the rollout status:

```
oc rollout status statefulset/<instance-name>
```

2. Verify the configuration by inspecting the environment variables inside the pod:

```
oc set env pod/<instance-name>-0 --list | grep LOG_LEVEL
```

Troubleshooting

- **Pod CrashLoopBackOff**: If the pods fail to start, check the logs of the Keycloak operator to see if there is an invalid configuration parameter passed in the `additionalOptions`.

```
oc logs deployment/keycloak-operator-controller-manager -n <operator-namespace>
```

- **Route Not Found**: If the route is missing, verify that the `hostname` field in your `Keycloak` CR is correctly specified and that the operator has sufficient permissions to create OpenShift Route objects.

For production environments, always refer to the **Configuring Red Hat build of Keycloak** documentation to ensure that your chosen configuration options are supported in the OpenShift containerized distribution.

Lab: Red Hat build of Keycloak on Red Hat OpenShift

Lab: Red Hat build of Keycloak on Red Hat OpenShift

In this lab, you will deploy the Red Hat build of Keycloak (RHBK) on a Red Hat OpenShift Container Platform cluster. You will utilize the Keycloak Operator to manage the lifecycle of your Keycloak instance.

Prerequisites

- Access to a Red Hat OpenShift cluster (version 4.12 or later).
- `oc` CLI tool authenticated with cluster-admin or sufficient privileges to create projects and operators.
- Knowledge of your OpenShift project namespace.

Task 1: Install the Keycloak Operator

The Keycloak Operator automates the deployment and management of Keycloak instances on OpenShift.

1. Log in to your OpenShift cluster via the CLI:

```
oc login -u <username> <api-url>
```

2. Create a new namespace for your Keycloak deployment:

```
oc new-project rhbk-lab
```

3. Create a `Subscription` to install the Keycloak Operator from the OperatorHub:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: keycloak-operator
```

```
  namespace: rhbk-lab
spec:
  channel: stable
  name: keycloak-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

Apply the configuration: `oc apply -f subscription.yaml`

Task 2: Deploy the Keycloak Instance

Once the operator is running, you must define the `Keycloak` Custom Resource (CR) to initiate the deployment.

1. Create a file named `keycloak-instance.yaml` with the following content:

```
apiVersion: k8s.keycloak.org/v2alpha1
kind: Keycloak
metadata:
  name: rhbk-instance
  namespace: rhbk-lab
spec:
  instances: 1
  image: registry.access.redhat.com/rhbk/keycloak-rhel9:26.6
  db:
    vendor: postgres
    host: keycloak-db
  hostname:
    hostname: keycloak-rhbk-lab.apps.example.com
  tls:
    certificateSecret:
      secretName: keycloak-tls-secret
```

2. Apply the instance configuration:

```
oc apply -f keycloak-instance.yaml
```

Task 3: Verify the Deployment

Confirm that the pods are running and the instance is accessible.

1. Check the status of the Keycloak pods:

```
oc get pods -n rhbk-lab
```

2. Verify that the Keycloak CR status is `Ready`:

```
oc get keycloak rhbk-instance -n rhbk-lab
```

Task 4: Explore the Admin Console

Once the status confirms the instance is ready, retrieve the initial credentials generated by the operator.

1. Retrieve the initial admin password from the secret created by the operator:

```
oc get secret rhbk-instance-initial-admin -n rhbk-lab -o
jsonpath='{.data.password}' | base64 --decode
```

2. Access the URL provided in your `hostname` configuration to reach the Admin Console and log in with the username `admin` and the retrieved password.

Troubleshooting

- **Pod CrashLoopBackOff**: Check the logs using `oc logs <pod-name> -n rhbk-lab` to identify configuration errors or database connection issues.
- **Image Pull Errors**: Ensure your cluster has access to `registry.access.redhat.com` or that you have configured an ImagePullSecret if using a private mirror.
- **Compatibility**: Ensure your OpenShift version (4.12, 4.14, 4.16, etc.) aligns with the supported matrix for the version of RHBK (e.g., 26.6.x) you are deploying.

Extending and Templating Keycloak

Extending and Templating Keycloak

Red Hat build of Keycloak provides flexibility to customize and extend core functionality to meet specific application and branding requirements:

- **UI Customization**: Modify login, registration, and account management templates to align with organizational branding and user experience standards.
- **Custom Adapters**: Extend client integration capabilities by implementing the `KeycloakAdapter` interface, allowing developers to create custom logic for authentication and session management.
- **Flow Customization**: While built-in authentication flows are immutable, you can alter their specific requirements and execution logic to satisfy unique security or business policies.

Customizing user interfaces (Login, Registration, Account Management)

Customizing user interfaces (Login, Registration, Account Management)

In the Red Hat build of Keycloak, themes define the look and feel of the user-facing pages. Customization allows organizations to align the authentication experience with their brand identity while maintaining a secure and consistent user journey across login, registration, and account management portals.

Understanding Keycloak Theming

Keycloak uses a template-based engine (FreeMarker) to render HTML. Instead of modifying core files, you create a custom theme directory that overrides specific templates, styles, or resources.

There are three primary areas of customization: * **Login Theme:** Controls the login, OTP, registration, and "forgot password" pages. * **Account Theme:** Controls the user profile and account management console. * **Email Theme:** Controls the look of automated emails sent to users (e.g., password reset links).

Dynamic User Profile Customization

Modern Red Hat build of Keycloak versions prioritize dynamic configuration over static template changes. You can modify the user experience without deep-diving into code by using the User Profile feature.

- **Attribute Validation:** Define validation rules on a per-attribute basis using built-in validators.
- **Dynamic Rendering:** Keycloak dynamically renders forms (registration, update profile, account console) based on your schema definitions.
- **Admin Console Integration:** Custom attributes are automatically rendered in the

Administration Console, ensuring that your user management interface stays in sync with your defined attributes.

Guided Exercise: Customizing your Login Theme

In this exercise, you will create a basic custom theme to alter the appearance of your login page.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- Access to the file system where Keycloak is installed.

Step 1: Create the Directory Structure

Navigate to your Keycloak installation directory and create the following structure:

```
mkdir -p themes/mycompany/login
```

Step 2: Define the Theme Metadata

Create a `theme.properties` file inside `themes/mycompany/login/` to tell Keycloak to extend the base theme.

```
# themes/mycompany/login/theme.properties
parent=base
import=common/default
```

Step 3: Override a Template

To change the login page layout, copy the `login.ftl` file from the base theme and place it in your custom directory.

```
cp themes/base/login/login.ftl themes/mycompany/login/
```

Open `themes/mycompany/login/login.ftl` and modify the HTML structure as needed. For example, adding a custom header or changing CSS classes.

Step 4: Apply the Theme

1. Log in to the **Admin Console**.
2. Select your **Realm**.
3. Navigate to **Realm settings** > **Themes**.
4. In the **Login theme** dropdown, select `mycompany`.
5. Click **Save**.

Best Practices for Customization

- **Use the `parent` property:** Always extend the `base` or `keycloak` theme. This ensures that you inherit essential functionality (like security fixes and hidden form fields) while only overriding the specific files you need to change.
- **Avoid hardcoding:** Whenever possible, use message bundles (`messages_en.properties`) for text strings to maintain multi-language support.
- **Leverage User Profile configuration:** Before editing raw FTL files for user registration forms, check the "User Profile" tab in the Admin Console. Many structural changes to forms can be handled via configuration rather than template modification.
- **Caching:** In development mode, you may want to disable theme caching in `conf/keycloak.conf` by setting `http-relative-path=/` and ensuring caching is disabled so changes reflect immediately without restarting the server.

Extending Keycloak functionality

Extending Red Hat build of Keycloak Functionality

Red Hat build of Keycloak is designed with a modular architecture that allows developers to extend its core capabilities to meet specific business requirements. While the platform provides comprehensive built-in features, such as predefined authentication flows (refer to section 8.3.1), certain advanced use cases may require custom logic.

This section covers how to extend the platform, specifically focusing on the client-side extensibility via custom adapters.

Understanding Keycloak Extensibility

Extending Red Hat build of Keycloak generally falls into two categories:

- **Server-side Extensions:** Customizing identity providers, user storage providers, or authentication SPIs (Service Provider Interfaces) to integrate with legacy systems or custom databases.
- **Client-side Extensions:** Leveraging the Keycloak JS adapter to control how applications interact with the Keycloak server, including custom login handling.

Customizing Client Interaction with Adapters

For web applications using the `keycloak-js` adapter, developers can override default browser-based behaviors by implementing the `KeycloakAdapter` interface. This is useful when your application environment (e.g., specific mobile wrappers or non-standard browser environments) requires a non-standard way to handle redirects or token storage.

Implementing a Custom Adapter

To create a custom adapter, you must implement the `KeycloakAdapter` interface. Using TypeScript ensures that all required methods—such as `login`, `logout`, and `register`—are correctly defined.

Example: Defining a Custom Adapter in TypeScript

```
import Keycloak, { KeycloakAdapter } from 'keycloak-js';

// Implement the 'KeycloakAdapter' interface
const MyCustomAdapter: KeycloakAdapter = {
    async login(options) {
        // Custom implementation for login
        console.log("Custom login logic triggered");
    },
    async logout(options) {
        // Custom implementation for logout
    },
    async register(options) {
        // Custom implementation for registration
    },
    // The other required methods must be implemented here...
};
```

Applying the Adapter

Once your adapter is defined, pass it to the `init` method during the initialization of your Keycloak instance. This instructs the client-side library to delegate its core lifecycle actions to your custom implementation.

Example: Initializing Keycloak with a Custom Adapter

```
const keycloak = new Keycloak({
    url: "http://keycloak-server",
    realm: "my-realm",
    clientId: "my-app"
});

await keycloak.init({
    adapter: MyCustomAdapter,
});
```

Hands-on Activity: Configuring a Custom Client Adapter

In this activity, you will prepare your application to use a custom adapter to log specific authentication events to your browser console.

1. **Prerequisites**: Ensure you have a basic project initialized with `keycloak-js`.
2. **Step 1: Create the Adapter**: Create a new file named `custom-adapter.ts`. Implement the `KeycloakAdapter` interface. For the `login` method, add a `console.log` statement that prints "Authentication initiated via Custom Adapter."
3. **Step 2: Integrate the Adapter**: In your main application file, import `MyCustomAdapter` and update your `keycloak.init()` call to include the `adapter` property.

4. **Step 3: Verification**:
    - Compile your TypeScript code.
    - Launch your application in a browser.
    - Open the browser's Developer Tools (Console tab).
    - Trigger the `keycloak.login()` function and observe if your custom log message appears before the redirect occurs.

Troubleshooting Tips

- **Incomplete Implementation**: If you fail to implement all methods required by the `KeycloakAdapter` interface, your TypeScript compiler will throw an error. Ensure all interface members are present.
- **Adapter Conflicts**: If `keycloak-js` behavior seems inconsistent, verify that your custom adapter is not overriding standard browser navigation methods in a way that breaks the OIDC/OAuth 2.0 flow. Always call the super-methods or replicate the original logic if you need to maintain base functionality.

Guided Exercise: Templating Keycloak

Guided Exercise: Templating Keycloak

In this exercise, you will customize the look and feel of the Red Hat build of Keycloak by overriding the default themes. Keycloak uses a FreeMarker-based templating engine that allows you to modify login pages, account management consoles, and email templates to match your organization's branding.

Prerequisites

- A running instance of Red Hat build of Keycloak 26.6.
- Access to the file system where the Keycloak server is installed.
- A basic understanding of HTML and CSS.

Step 1: Locating the Themes Directory

By default, Keycloak stores its themes in the `themes/` directory within the server installation path. You should never modify these files directly, as your changes will be overwritten during server upgrades. Instead, you will create a custom theme directory.

1. Navigate to your Keycloak installation directory.
2. Create a new directory structure for your custom theme:

```
mkdir -p themes/my-company/login/resources/css
mkdir -p themes/my-company/login/resources/img
```

Step 2: Creating the Theme Configuration

Keycloak requires a `theme.properties` file to recognize your new theme.

1. Create a file named `theme.properties` inside the `themes/my-company/login/` directory.
2. Add the following content to inherit from the base Keycloak theme:

```
parent=base
import=common/keycloak
```

Step 3: Overriding a Template

To change the structure of the login page, you must copy the template file from the base theme and modify it.

1. Copy the login template from the Keycloak base directory to your custom directory:

```
cp themes/base/login/login.ftl themes/my-company/login/
```

2. Open `themes/my-company/login/login.ftl` in your editor.
3. Locate the header section and add a custom welcome message or your company logo reference.

Step 4: Applying Custom Styles

You can override the CSS to change colors, fonts, and layout.

1. Create a file named `styles.css` in `themes/my-company/login/resources/css/`.
2. Add your custom CSS, for example, to change the background color:

```
body {
    background-color: #f0f0f0 !important;
}
```

3. Update your `theme.properties` file to include your new stylesheet:

```
styles=css/styles.css
```

Step 5: Enabling the Theme in the Admin Console

Once the files are in place, you must instruct Keycloak to use your new theme.

1. Open the **Admin Console** and log in.
2. Select your **Realm** from the dropdown menu in the top-left corner.
3. In the left navigation menu, go to **Realm settings**.

4. Click the **Themes** tab.
5. In the **Login theme** dropdown, select `my-company`.
6. Click **Save**.

Step 6: Verification

1. Open a new Incognito/Private browser window.
2. Navigate to your login page (e.g., `http://localhost:8080/realms/<your-realm>/account`).
3. Verify that your custom background color and any changes made to `login.ftl` are rendered correctly.

Troubleshooting

- **Changes not appearing:** If you are in development mode, ensure you have disabled theme caching in `conf/keycloak.conf`:

  ```
  http-relative-path=/
  # Disable theme caching for development
  kc.cache.theme=off
  ```

- **Syntax Errors:** If the page fails to render, check the server logs; FreeMarker errors will be clearly indicated with line numbers.

Comprehensive Review

Comprehensive Review

The Comprehensive Review module serves as the final assessment of the Red Hat build of Keycloak training curriculum. It reinforces technical proficiency by consolidating core security concepts and practical configuration skills through the following activities:

- **Conceptual Consolidation:** A holistic recap of identity management principles, including single sign-on (SSO) standards (OIDC, OAuth 2.0, SAML 2.0) and architectural foundations.
- **Security Validation:** Reviewing essential security mechanisms, such as credential isolation, token handling, and authentication flow management.
- **Applied Assessment:** A concluding laboratory exercise that requires the integration of installation, configuration, and administrative tasks to verify overall mastery of the Red Hat build of Keycloak environment.

Review of key security concepts and configurations

Review of key security concepts and configurations

This section provides a summary of the fundamental security concepts and administrative configurations required to maintain a hardened Red Hat build of Keycloak environment.

Security Concepts Review

To effectively secure applications using Red Hat build of Keycloak, you must understand the interaction between identity standards and the server's architectural components.

- **Identity Protocols:** Keycloak acts as an Identity Provider (IdP) supporting:
- **OIDC (OpenID Connect):** An identity layer on top of the OAuth 2.0 protocol, allowing clients to verify the identity of the end-user.
- **OAuth 2.0:** The industry-standard protocol for authorization, enabling applications to obtain limited access to user accounts.
- **SAML 2.0:** An XML-based standard for exchanging authentication and authorization data between parties.
- **Credential Isolation:** Keycloak ensures that user credentials remain within the Identity Provider, protecting applications from handling sensitive user data directly.
- **Token Mechanisms:** Security is enforced through short-lived access tokens, refresh tokens, and ID tokens, which facilitate secure communication between the client, the Keycloak server, and the protected resource.

Configuration Checklist

When deploying or auditing your Keycloak environment, ensure the following configurations are addressed:

Configuration Area	Security Focus
Realms	Ensure each application or department is isolated within its own realm to enforce separation of duties.
Authentication Flows	Review the default browser and direct grant flows. Implement Multi-Factor Authentication (MFA) for high-privilege accounts.
Client Settings	Use strict redirect URIs. Never use wildcard URIs in production environments. Ensure "Standard Flow" is preferred over "Implicit Flow" for modern applications.
User Federation	When syncing with LDAP or Active Directory, ensure that the connection is secured via LDAPS to protect credentials in transit.

Lab: Comprehensive Security Audit

In this activity, you will perform a security review of a pre-configured Red Hat build of Keycloak environment.

1. **Step 1: Audit Token Lifespans**
   - Log in to the Admin Console.
   - Navigate to **Realm Settings** > **Tokens**.
   - Review the **Access Token Lifespan**. Ensure it is set to a reasonable duration (e.g., 5 minutes) to minimize risk if a token is intercepted.
2. **Step 2: Verify Client Configuration**
   - Navigate to **Clients**.
   - Select a configured client (e.g., your SPA).
   - Verify that the **Valid Redirect URIs** contain specific, non-wildcard entries.
   - Ensure **Client Authentication** is enabled if the client is capable of keeping a secret.
3. **Step 3: Review Authentication Policies**
   - Go to **Authentication** > **Policies**.
   - Ensure that "User Registration" is disabled if you are using an external User Federation (e.g., LDAP) to prevent unauthorized local accounts.
4. **Step 4: Check Provider Connectivity**
   - If using an Identity Broker or User Federation, verify that the connection settings use `ldaps://` or secure transport layers to ensure encrypted communication between Keycloak and the external directory.

Regularly reviewing these configurations is critical for maintaining the security posture of your organization's identity infrastructure. Always document deviations from the default security templates for compliance reporting.

Lab: Install and Configure Red Hat build of Keycloak

Lab: Install and Configure Red Hat build of Keycloak

This lab focuses on setting up a Red Hat build of Keycloak instance for development and testing purposes. You will use `podman` to deploy a containerized instance and perform initial configuration via the Admin Console.

Objectives

- Deploy Red Hat build of Keycloak in development mode.
- Initialize the administrative account.
- Access and explore the Admin Console.

Prerequisites

- A system with `podman` installed.
- Access to the Red Hat Ecosystem Catalog to pull the `rhbk/keycloak-rhel9` image.
- Network access to port 8080.

Activity: Deploying Keycloak in Development Mode

The development mode is optimized for rapid prototyping. It disables strict HTTPS requirements and defaults to an in-memory database.

Development mode is intended strictly for local testing. Never use these default settings or this startup mode in a production environment as it lacks enterprise-grade security configurations.

1. **Start the Keycloak container:** Execute the following command in your terminal to pull the image and start the service:

```
podman run --name mykeycloak -p 127.0.0.1:8080:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin \
  -e KC_BOOTSTRAP_ADMIN_PASSWORD=change_me \
  registry.redhat.io/rhbk/keycloak-rhel9:26.6 \
  start-dev
```

1. **Verify Initialization:** Watch the container logs. Once you see the message `Keycloak <version> started in (development) mode`, the server is ready to accept connections.

Activity: Exploring the Admin Console

Once the server is running, you must configure the initial realm settings.

1. **Access the Console:** Open your web browser and navigate to `http://localhost:8080`.

2. **Authenticate:** Click the **Administration Console** link. Log in using the credentials defined in the deployment command:
    - **Username:** `admin`
    - **Password:** `change_me`
3. **Explore the Interface:**
    - **Realms:** Observe the default `master` realm. The master realm is used to manage the Keycloak instance itself and should be reserved for administrative tasks.
    - **Clients:** Navigate to the Clients section to see the default integrated clients.
    - **Users:** Note how users are currently managed within the internal database.

Troubleshooting Guidance

- **Port Conflicts:** If `8080` is already in use on your host machine, modify the port mapping in the `podman run` command (e.g., `-p 127.0.0.1:8081:8080`).
- **Container Persistence:** Since development mode often uses an in-memory database, restarting the container will wipe all configuration changes. If you need to persist data, you must configure a persistent volume mapping to `/opt/keycloak/data`.
- **Login Failures:** If you are unable to log in, verify that the `KC_BOOTSTRAP_ADMIN_USERNAME` and `KC_BOOTSTRAP_ADMIN_PASSWORD` environment variables were passed correctly during the container creation.

Review

After completing these steps, you have successfully: * Provisioned a containerized Red Hat build of Keycloak. * Secured the initial administrative access. * Verified the connectivity and basic functionality of the Admin Console.

Appendix

FIXME: Content for Appendix…

Resources

Download Course PDF